School of Computer Science Engineering & Information Systems

Department of Software & Systems Engineering

ISWE406P – Agile Development Process and Devops Lab

ASSESSMENT -2

NAME : SARAVANAN.R

REGISTER NO : 23MIS0460

FACULTY NAME : DR . RAMALINGAM M

Experiment 1: Jenkins Familiarization

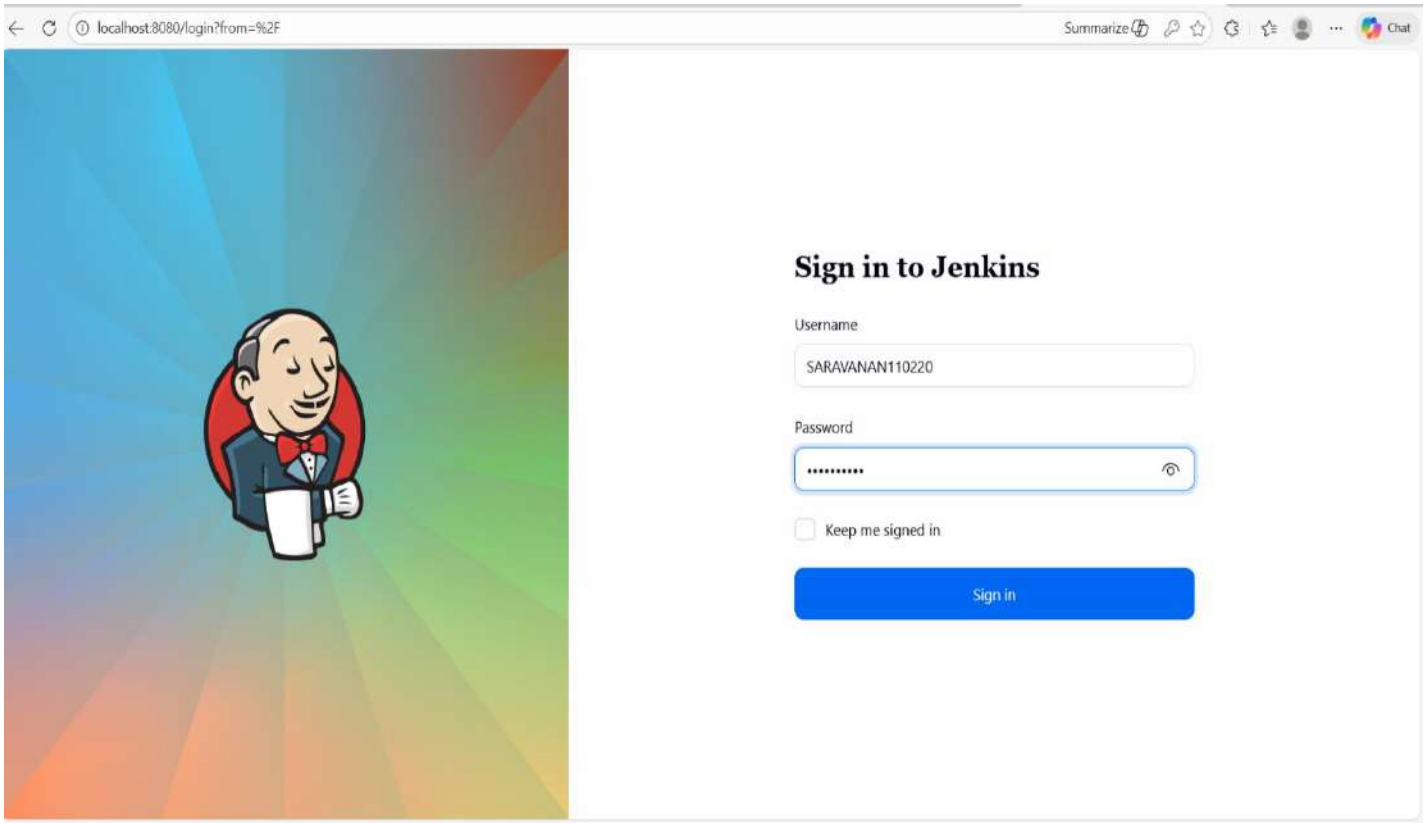
Objective:

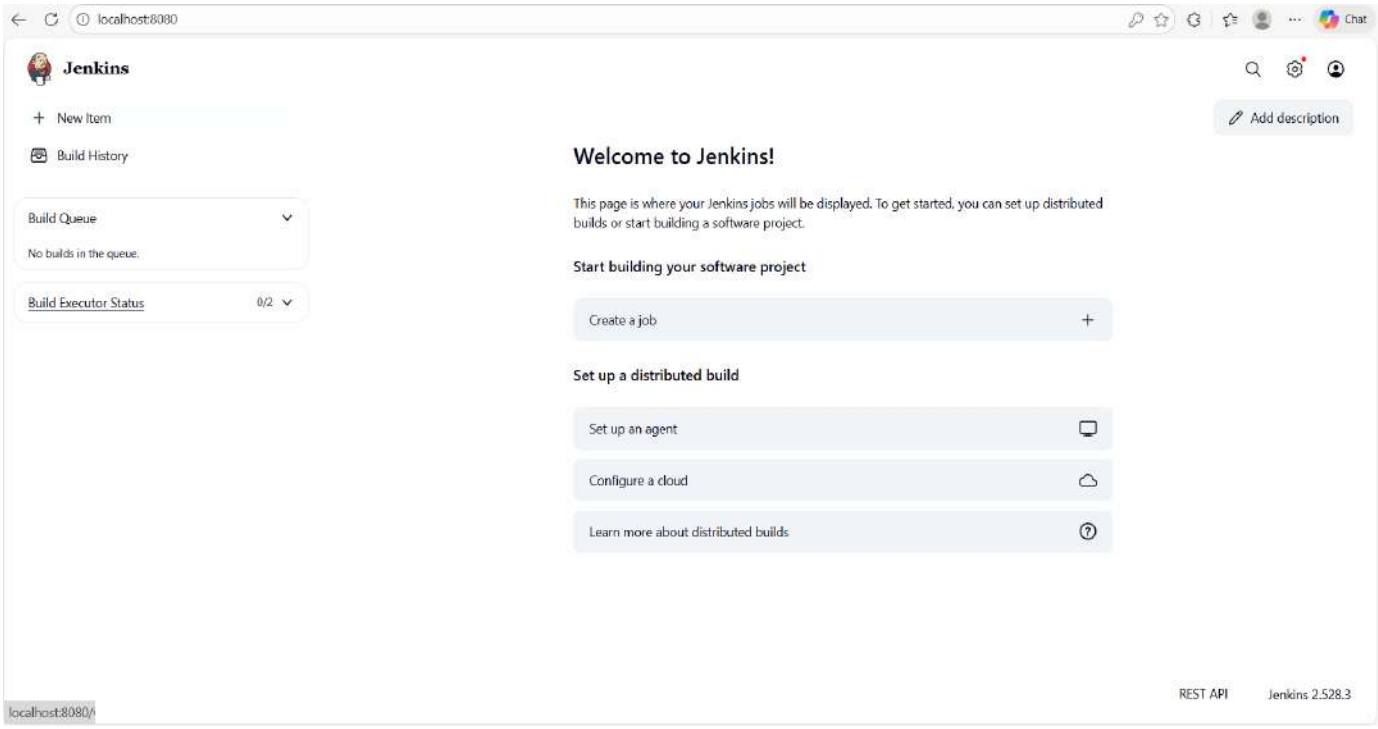
To start Jenkins server and explore the Jenkins dashboard and configuration options.

Steps Performed:

1. Verified Java installation using java -version.
2. Started Jenkins using:
3. java -jar jenkins.war
4. Accessed Jenkins at http://localhost:8080.
5. Entered initial admin password.
6. Installed suggested plugins.
7. Created admin user account.
8. Explored Dashboard and Manage Jenkins options.

Result:





Jenkins was successfully installed and the user interface was explored.

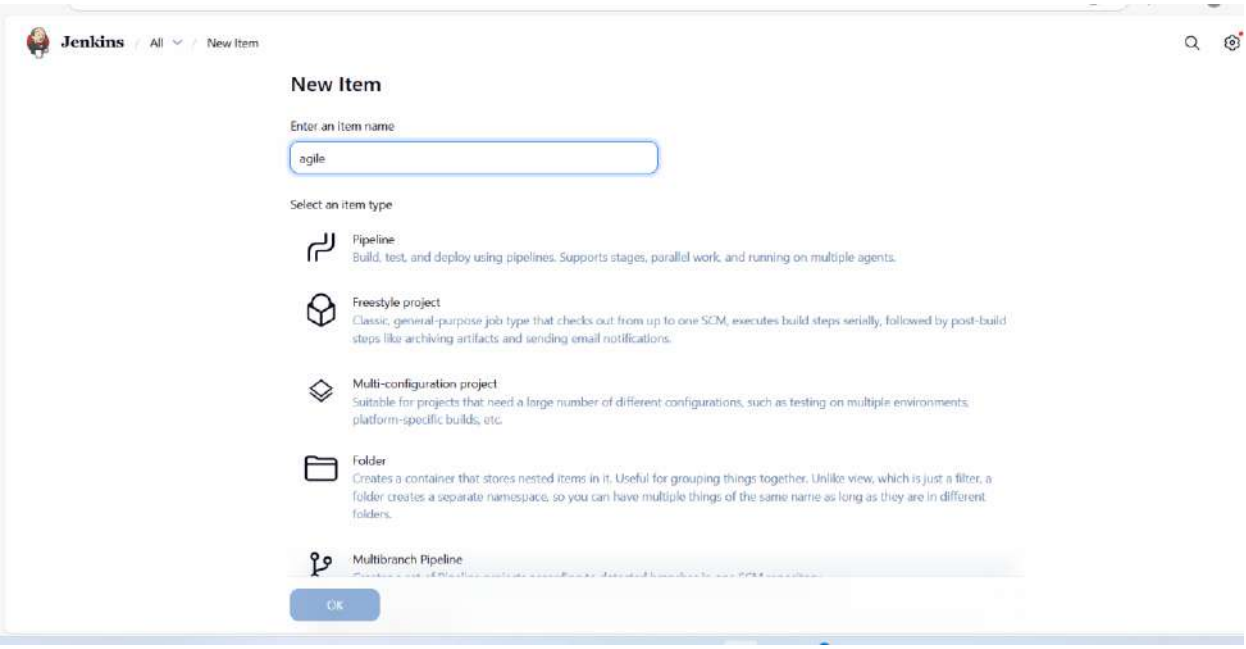
<u>Experiment 2:</u> Create First Freestyle Job

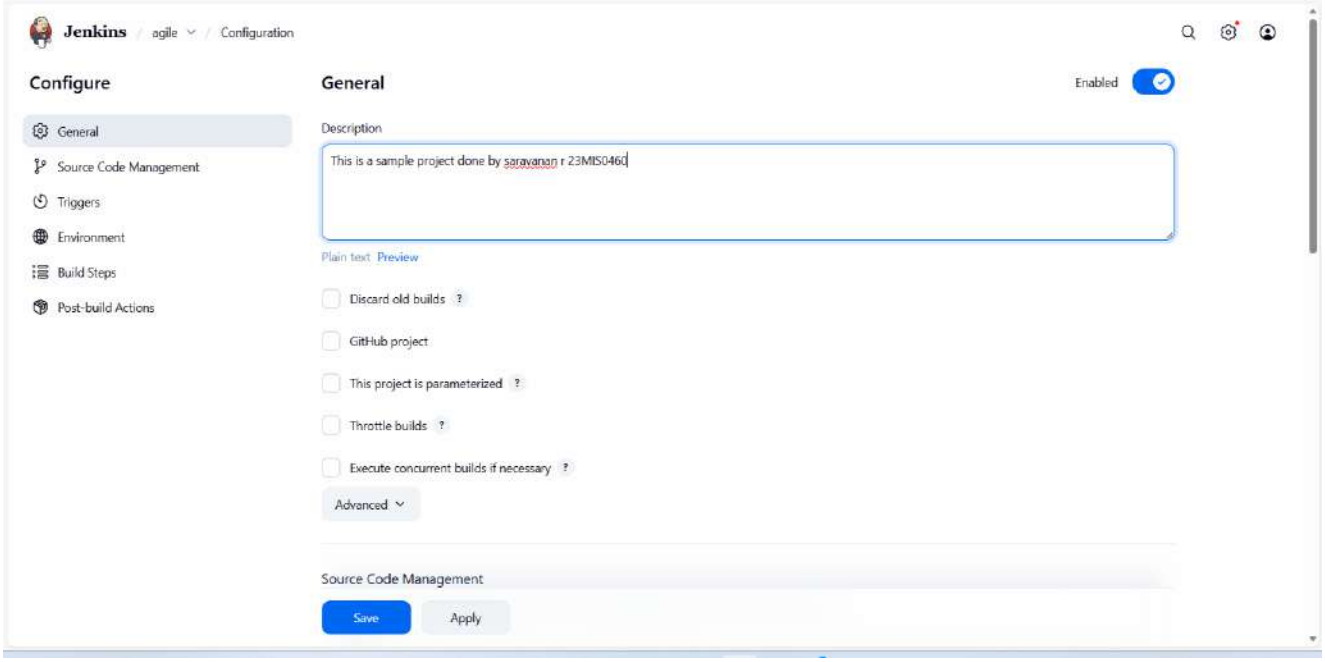
Objective:
To create and execute a simple Freestyle job in Jenkins.

Steps Performed:

1. Clicked New Item.
2. Entered job name as agile.
3. Selected Freestyle project.
4. Added description.
5. Added build step → Execute Windows batch command.
6. Entered command:
7. echo Hello Jenkins
8. Clicked Save.
9. Clicked Build Now.
10. Viewed Console Output.

<u>Output:</u>
The console displayed:





Jenkins / agile / Configuration

Configure

- General
- Source Code Management
- Triggers
- Environment
- Build Steps
- Post-build Actions

Build Steps

Automate your build process with ordered tasks like code compilation, testing, and deployment.

Execute Windows batch command

Command

See the list of available environment variables

```
echo Hello Jenkins
```

Advanced

+ Add build step

Post-build Actions

Define what happens after a build completes, like sending notifications, archiving artifacts, or triggering other jobs.

+ Add post-build action

Save Apply

Jenkins / agile

- Status
- Changes
- Workspace
- Build Now
- Configure
- Delete Project
- Rename

Builds

No builds

Build scheduled

localhost:8080

agile

Edit description

This is a sample project done by saravanan r 23MIS0460

Permalinks

REST API Jenkins 2.528.3

Jenkins / agile / #1

- Status
- Changes
- Console Output
- Edit Build Information
- Delete build '#1'
- Timings

#1 (Feb 6, 2026, 10:25:26 PM)

Add description Keep this build forever

Started by user SARAVANAN R

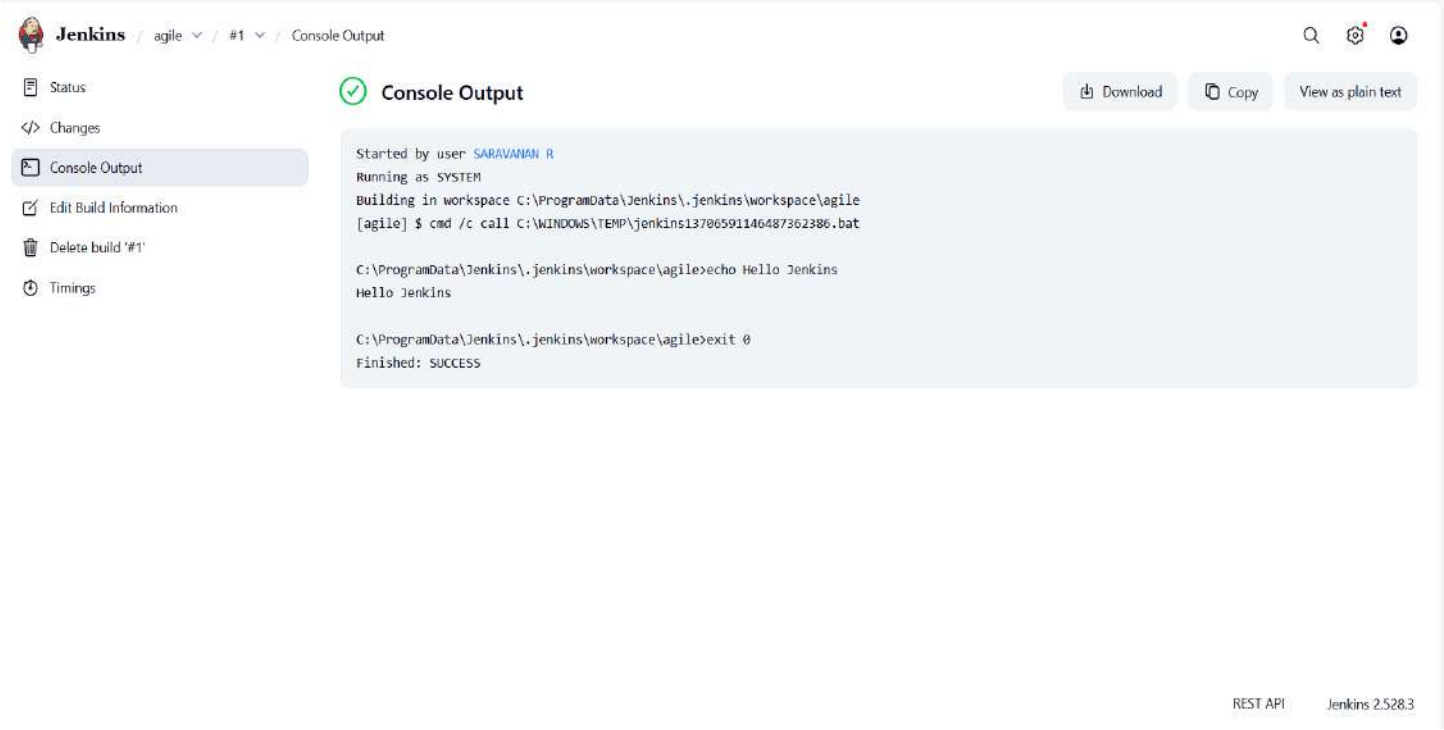
Started 42 sec ago
Took 0.34 sec

This run spent:

- 66 ms waiting;
- 0.34 sec build duration;
- 0.41 sec total from scheduled to completion.

No changes.

REST API Jenkins 2.528.3



Hello Jenkins
Finished: SUCCESS

Result:

Freestyle job was created and executed successfully.

EXPERIMENT – 3:

Building a C Program from GitHub Automatically Through Jenkins

Objective:

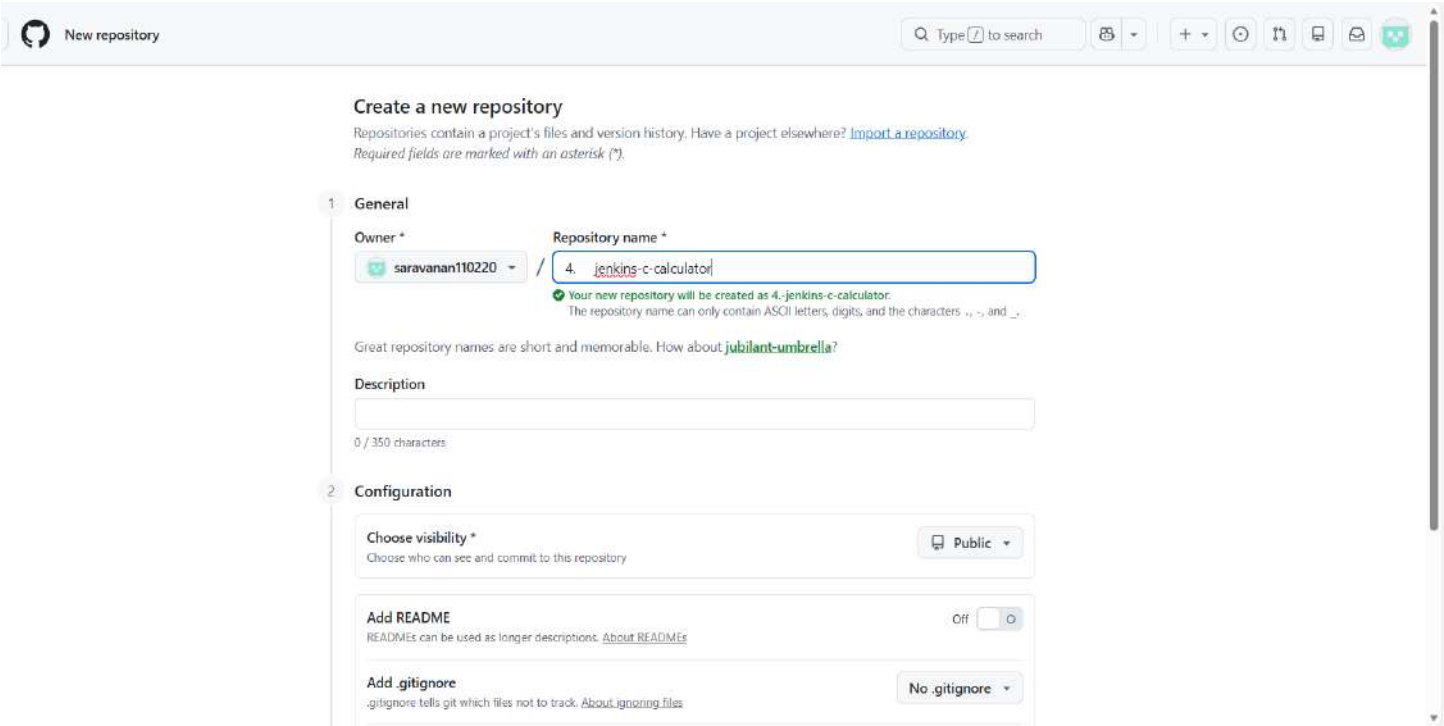
To integrate Jenkins with a GitHub repository and configure it to automatically compile and execute a C program whenever changes are pushed to the repository.

Tools Used :

- Jenkins Automation Server
- GitHub Account
- GCC Compiler (MinGW for Windows)
- Web Browser
- Windows Command Prompt

Step 1: Create a GitHub Repository

1. Login to GitHub.
2. Click on **New Repository**.
3. Enter repository name as:
4. jenkins-c-calculator
5. Select **Public**.
6. Click **Create Repository**.



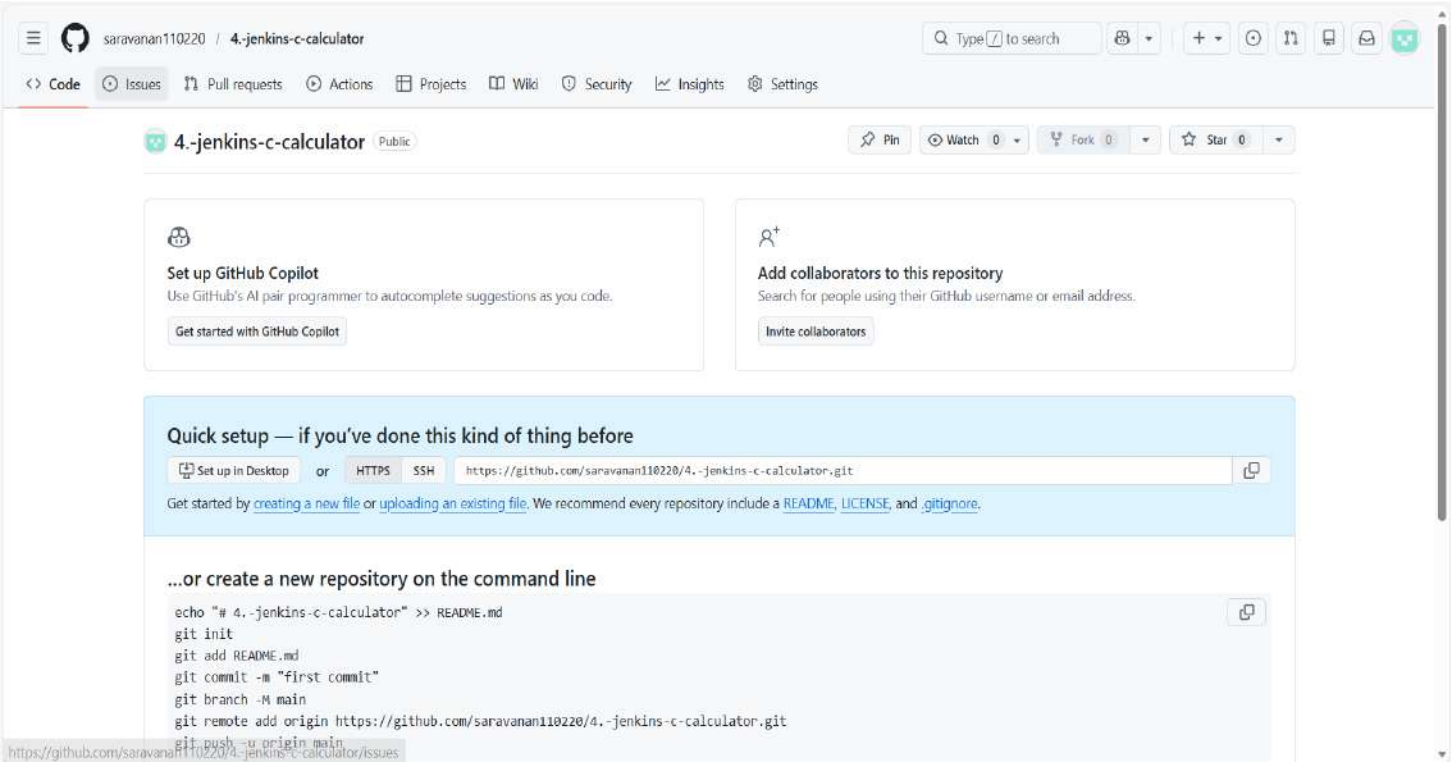
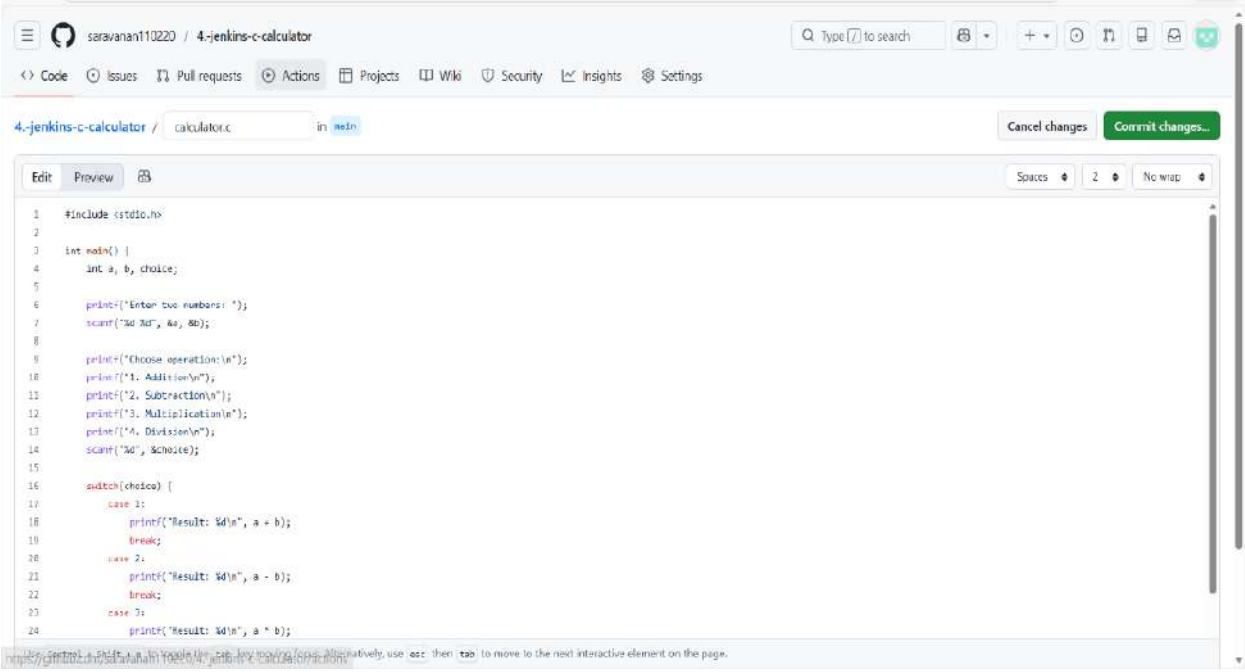


Figure 1: GitHub Repository Created Successfully

<u>Step 2:</u> Upload C Calculator Program

1. Click **Add file** → **Create new file**.
2. Enter file name:
3. calculator.c
4. Add the following C program:



5. **Click** Commit changes**.**

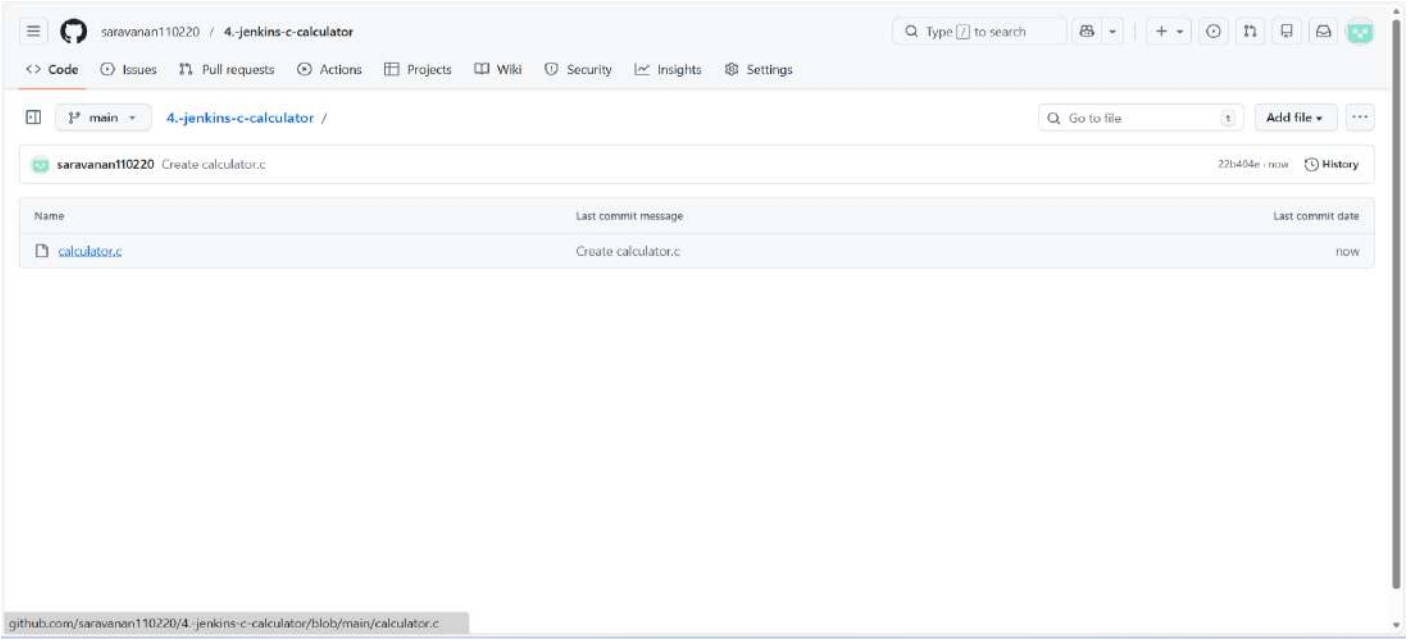


Figure 2: C Calculator Program Uploaded to GitHub

Step 3: Create a Jenkins Freestyle Job

1. Open Jenkins in browser:
2. http://localhost:8080
3. Click **New Item**.
4. Enter project name:

   Agile sample

5. Select **Freestyle Project**.
6. Click **OK**.

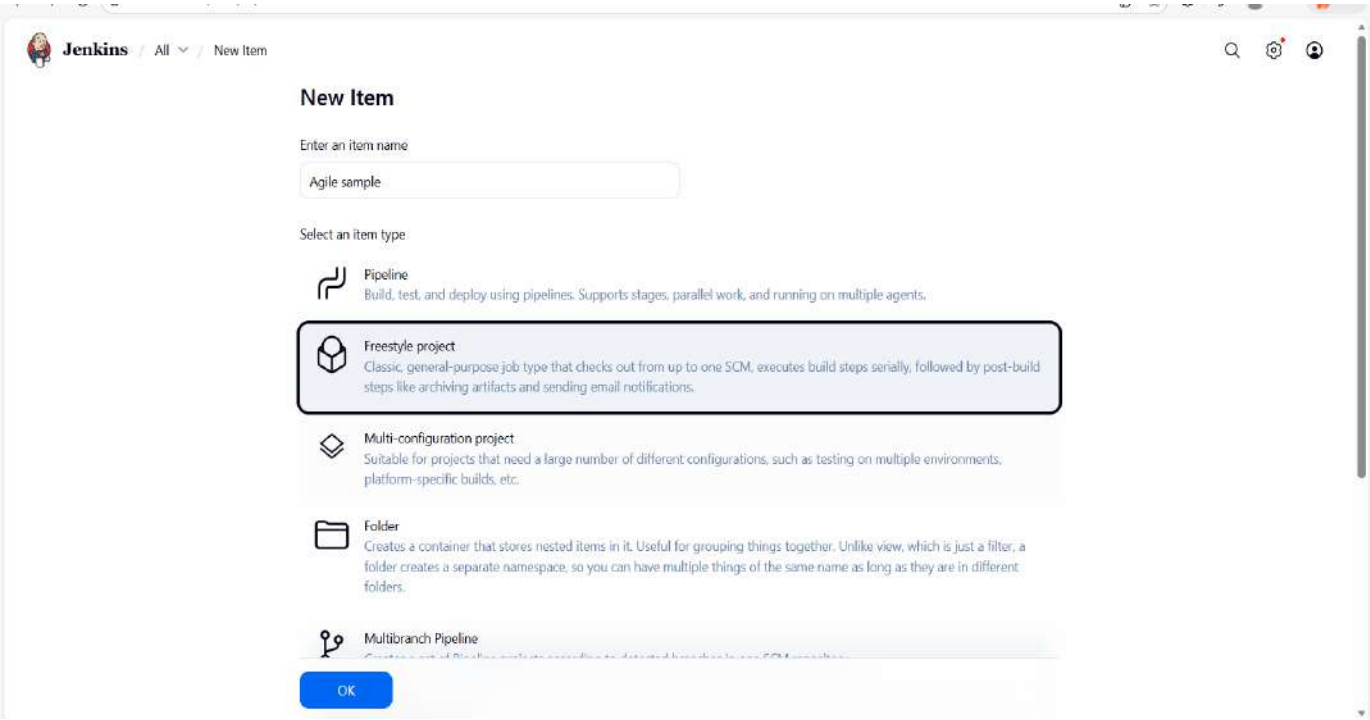


Figure 3: Creation of Freestyle Project in Jenkins

Step 4: Integrate GitHub Repository with Jenkins

1. Scroll to **Source Code Management**.
2. Select **Git**.
3. Enter repository URL:

   https://github.com/saravanan110220/4.-jenkins-c-calculator.git

4. Under Branches to Build, enter:

   */main

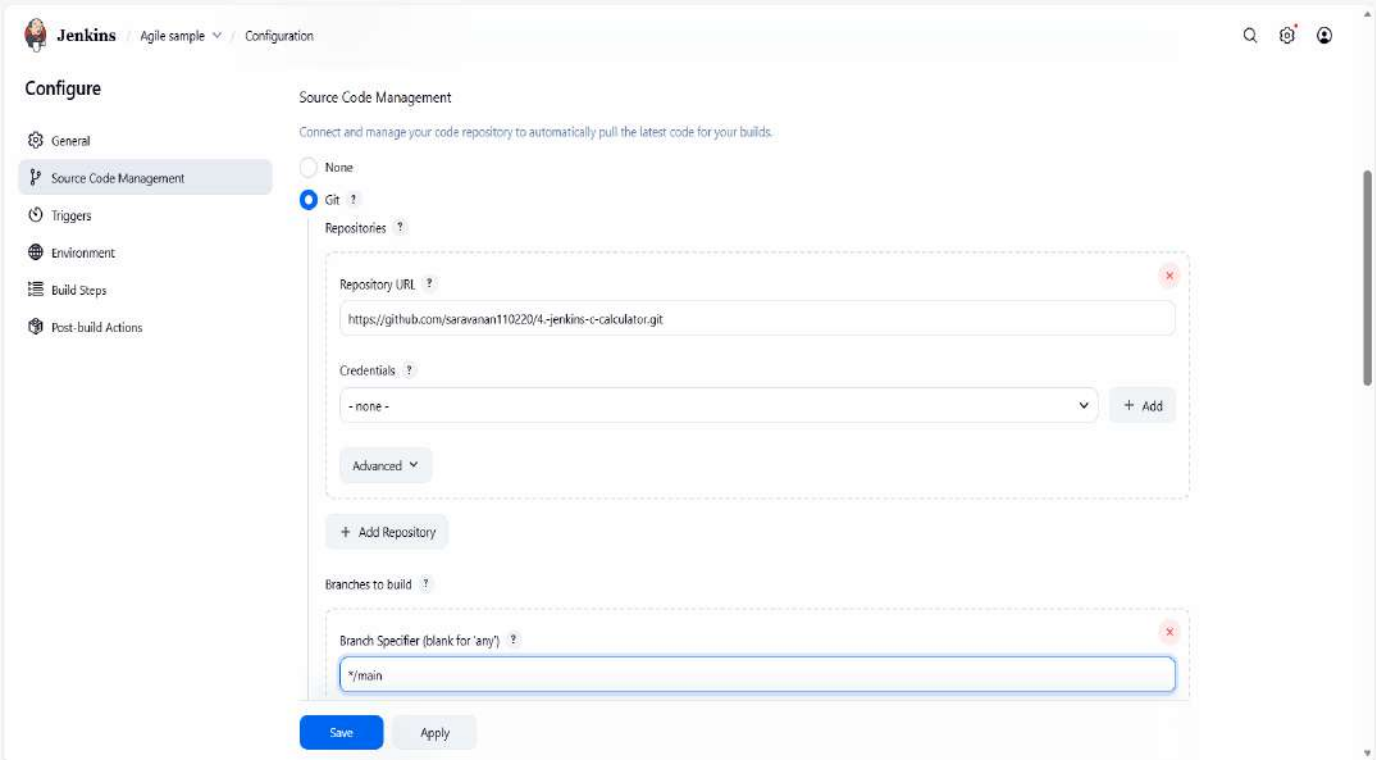


Figure 4: GitHub Repository Integrated in Jenkins

<u>Step 5:</u> Configure Build Step

1. Scroll to **Build Steps**.
2. Click **Add build step** → **Execute Windows batch command**.
3. Enter the following commands:

```
gcc calculator.c -o calculator.exe
echo 10 5 1 | calculator.exe
```

<u>Explanation:</u>

- First command compiles the C program.
- Second command executes the program with predefined input:
    - 10 and 5 are numbers.
    - 1 represents addition.

4. Click **Apply**

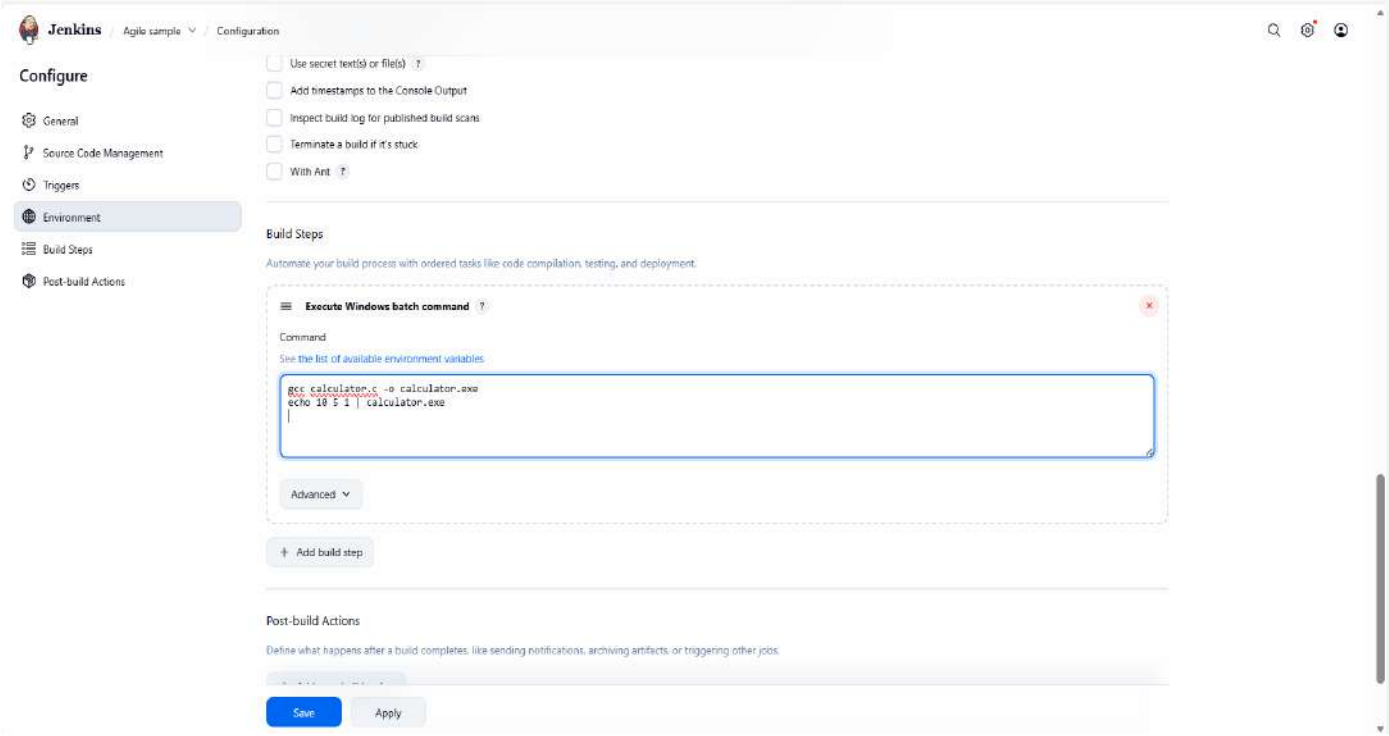


Figure 5: C Program Compilation and Execution Configured in Jenkins

<u>Step 6</u>: Enable Automatic Build Trigger

1. Scroll to **Build Triggers**.
2. Select **Poll SCM**.
3. Enter the cron schedule:

* * * * *

This checks the repository every one minute for changes.

4. Click **Save**.

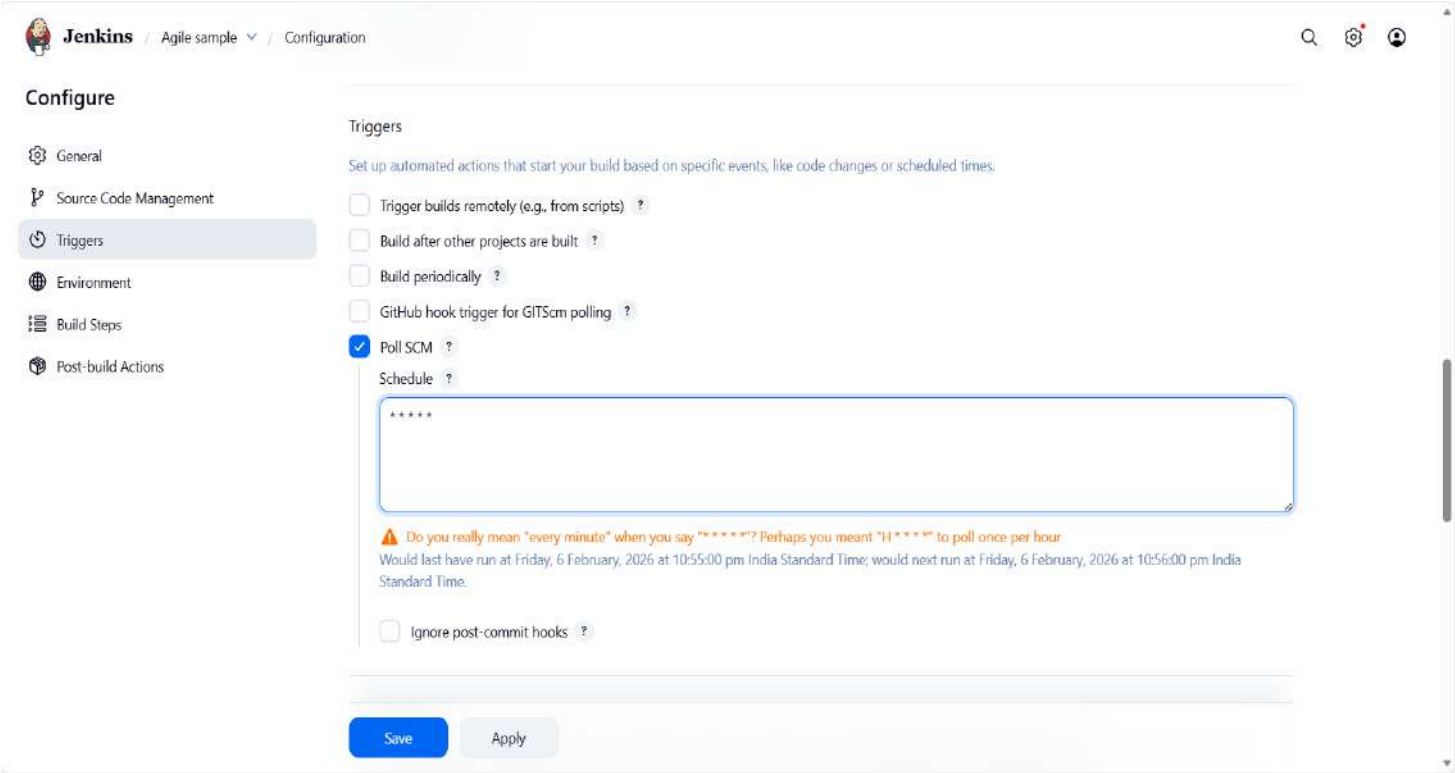


Figure 6: Poll SCM Enabled for Automatic Trigger

<u>Step 7:</u> Test Automatic Build

1. Go to GitHub.
2. Open calculator.c.
3. Edit the program.

For example, change:

printf("Result: %d\n", a + b);

To:

printf("Addition Result: %d\n", a + b);

4. Click **Commit changes**.
5. Wait for 1–2 minutes.

Jenkins will automatically detect the change and trigger a new build.

```c
scanf("%d %d", &a, &b);

printf("Choose operation:\n");
printf("1. Addition\n");
printf("2. Subtraction\n");
printf("3. Multiplication\n");
printf("4. Division\n");
scanf("%d", &choice);

switch(choice) {
    case 1:
        printf("Addition Result: %d\n", a + b);
        break;
```

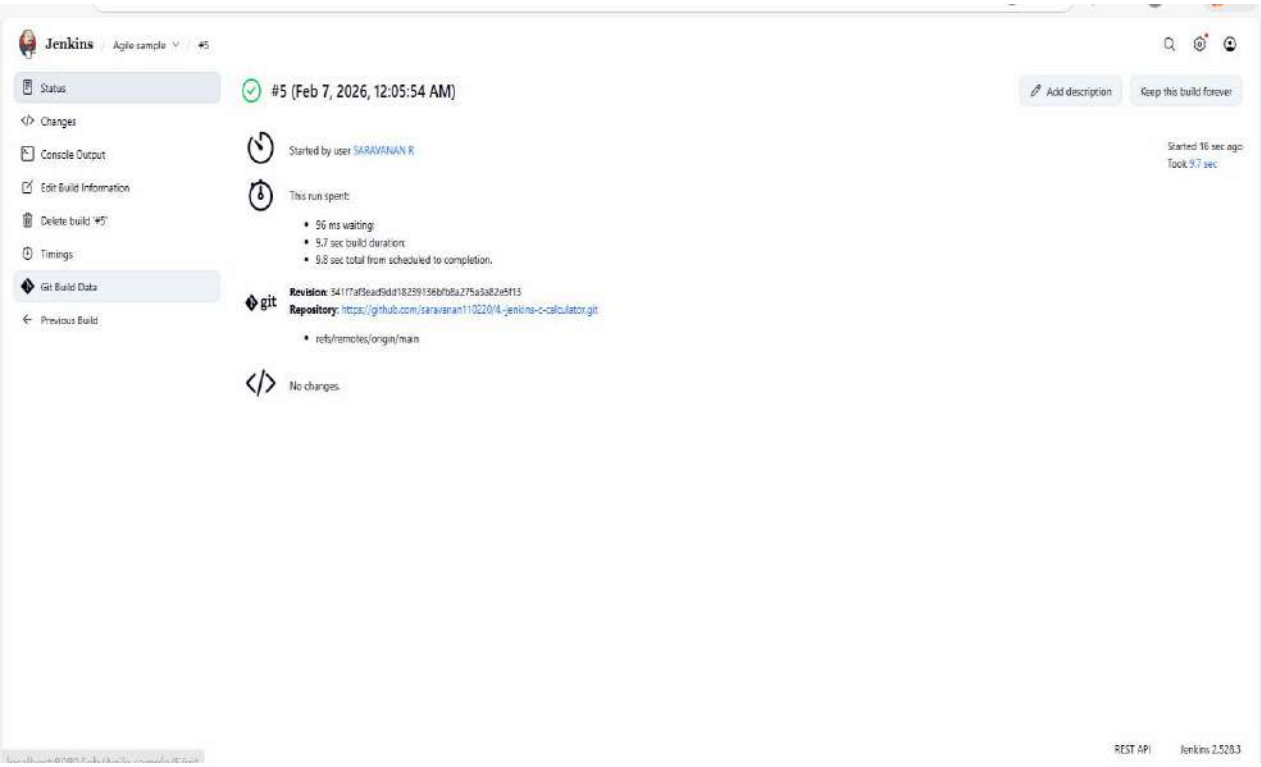


Figure 7: Automatic Build Triggered by GitHub Commit

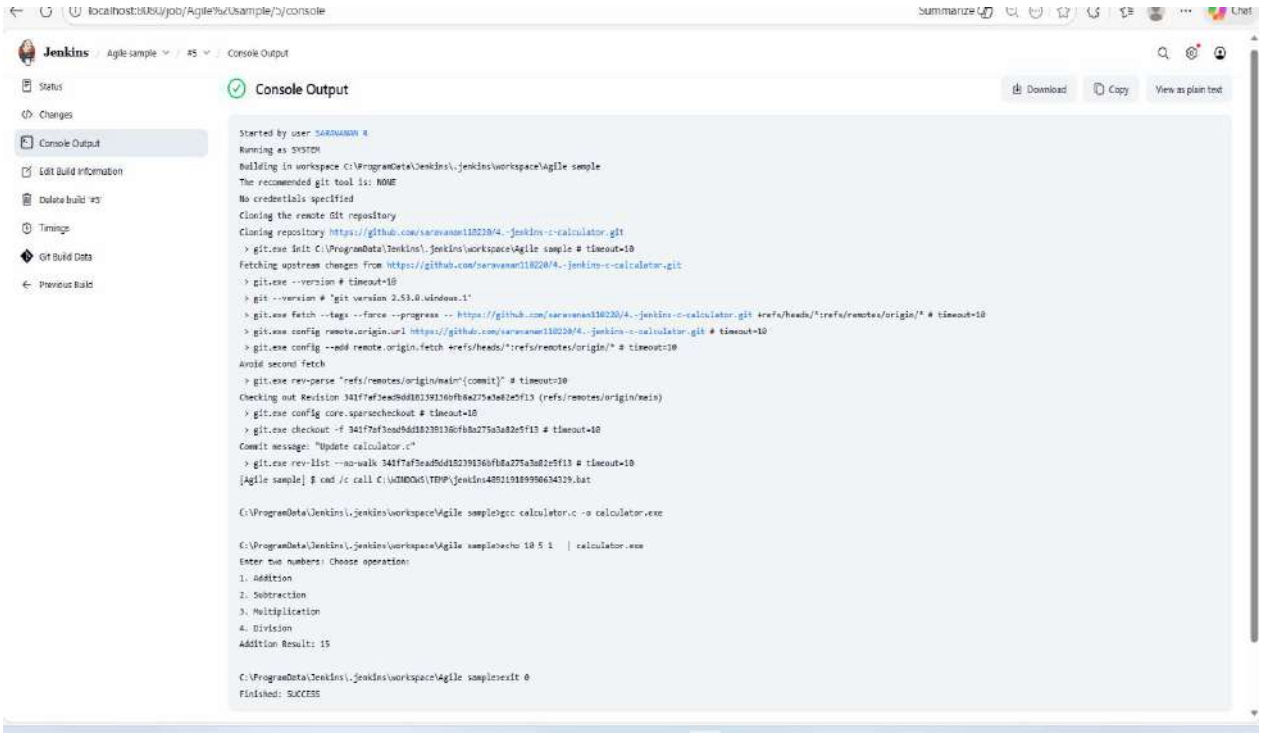


Figure 8: Console Output Showing Successful Execution

EXPERIMENT – 4 : Implementation of Basic Jenkins Pipelines

Objective:

To create and execute various Jenkins Declarative Pipeline jobs to perform operations such as code checkout, directory listing, message display, file creation, and file reading.

- **Pipeline Task 1: Checkout Code and Show Files**

Objective

To checkout source code from GitHub repository and display the list of files in the workspace using Jenkins Pipeline.

Steps

Step 1:

->Open Jenkins Dashboard.

Step 2:

->Click **New Item**.

Step 3:

->Enter name:

Pipeline-Task1

->Select:

✓ Pipeline
Click OK.

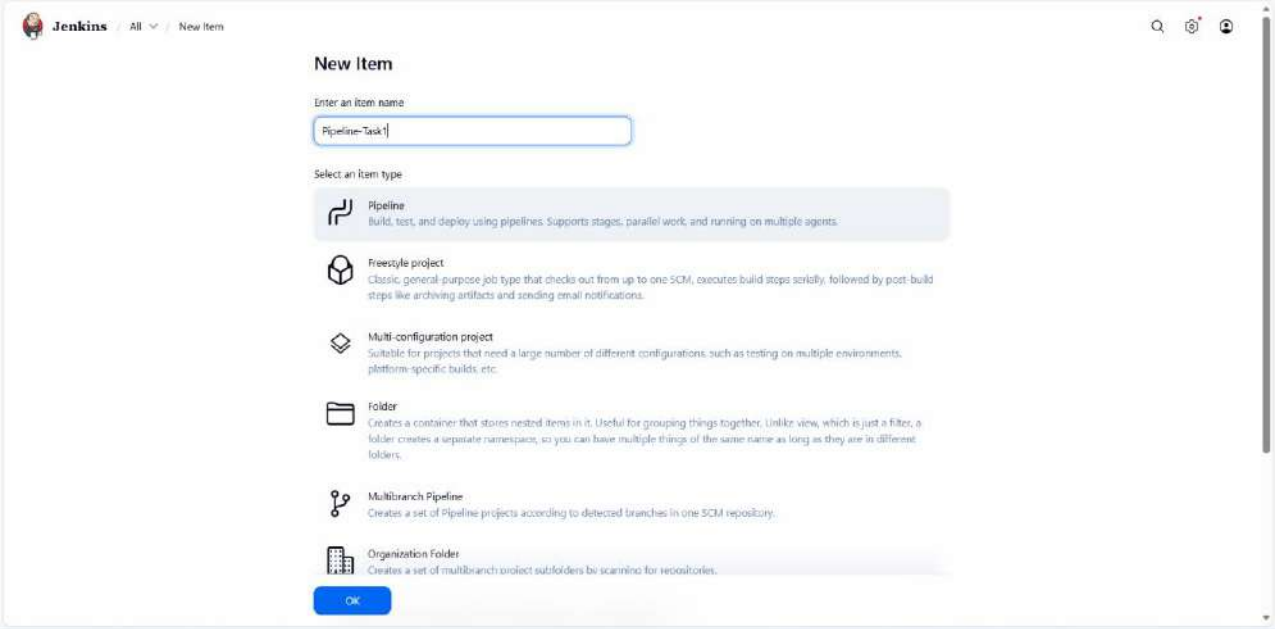


Figure 1: Creation of Pipeline Job

Step 4:

->Scroll to Pipeline section.

Select Definition:

✓ Pipeline Script

Step 5:

->Enter the following code:

```
 pipeline {
 agent any
 stages {
  stage('Checkout Code') {
   steps {
    git branch: 'main',
        url: 'https://github.com/saravanan110220/agilegithub.git'
   }
  }
  stage('Show Files') {
   steps {
    bat 'dir'
   }
  }
 }
```

}

Click Save.

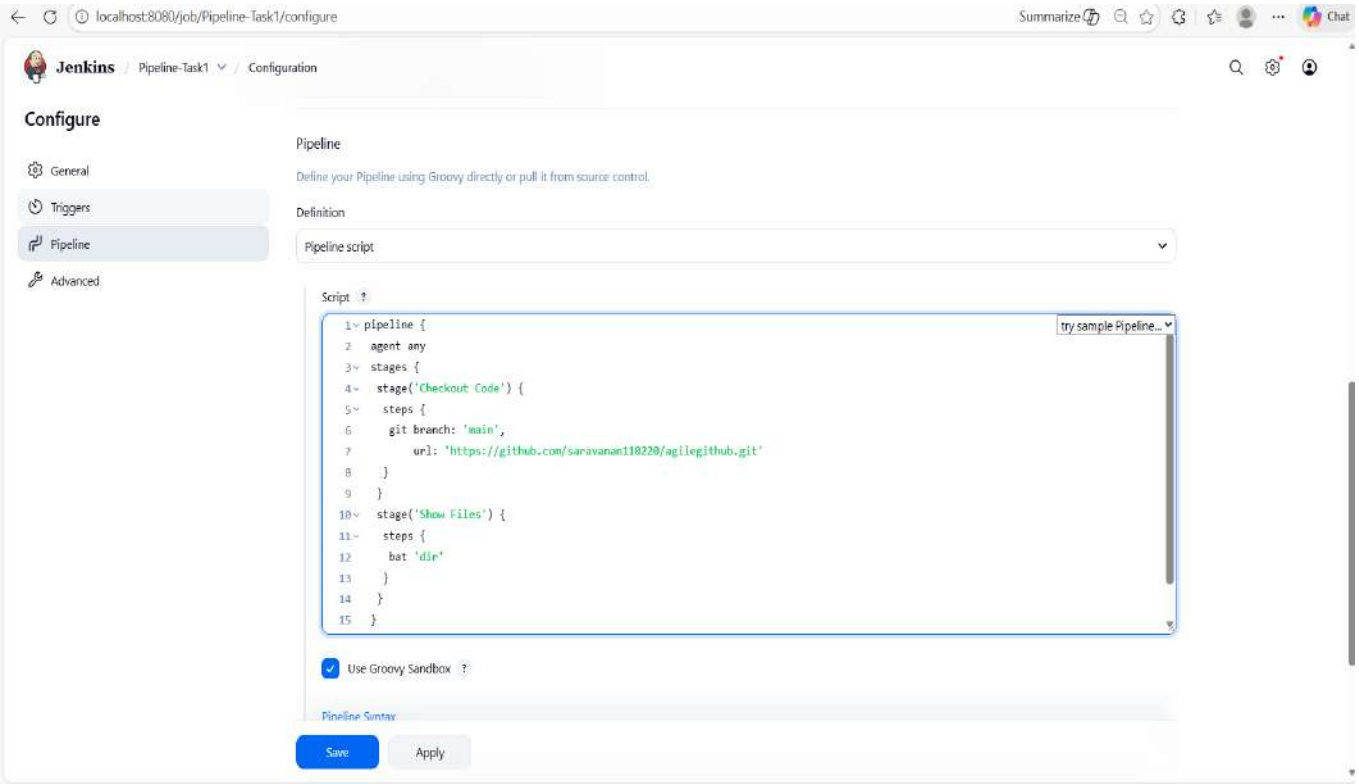
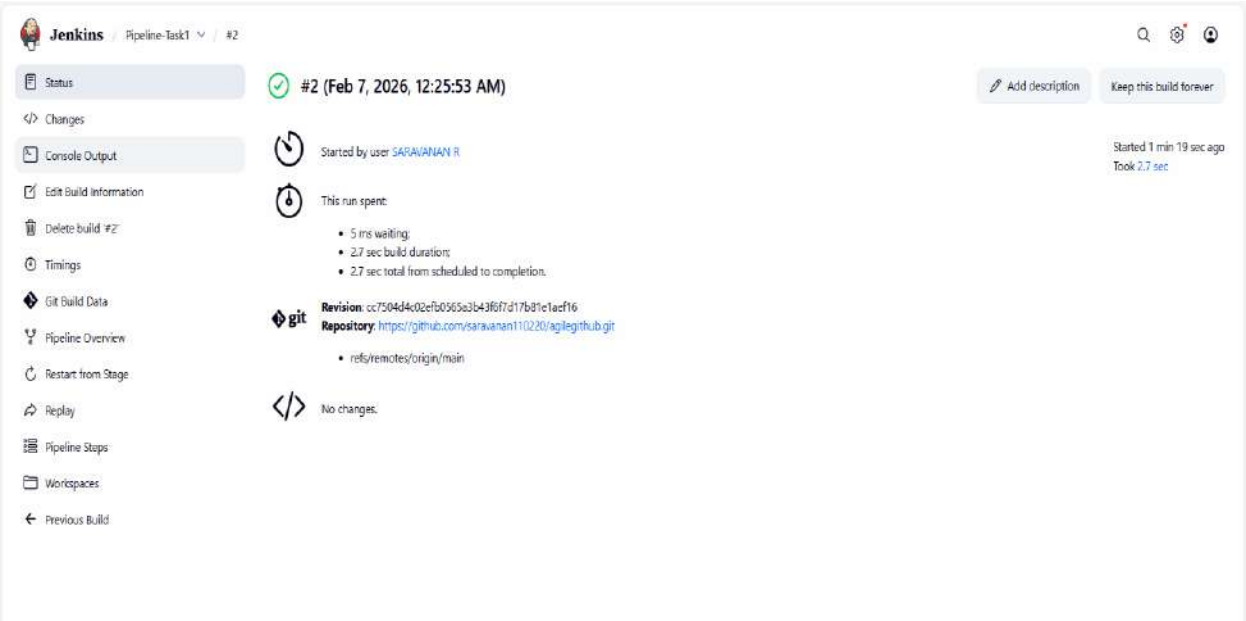


Figure 2: Pipeline Script for Checkout and File Listing

Step 6:

->Click **Build Now**.



Jenkins / Pipeline-Task1 / #2 / Console Output

- Status
- Changes
- Console Output
- Edit Build Information
- Delete build '#2'
- Timings
- Git Build Data
- Pipeline Overview
- Restart from Stage
- Replay
- Pipeline Steps
- Workspaces
- Previous Build

Console Output

Download | Copy | View as plain text

```
Started by user SARAVANAN R
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in C:\ProgramData\Jenkins\.jenkins\workspace\Pipeline-Task1
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Checkout Code)
[Pipeline] git
The recommended git tool is: NONE
No credentials specified
 > git.exe rev-parse --resolve-git-dir C:\ProgramData\Jenkins\.jenkins\workspace\Pipeline-Task1\.git # timeout=10
Fetching changes from the remote Git repository
 > git.exe config remote.origin.url https://github.com/saravanan110220/agilegithub.git # timeout=10
Fetching upstream changes from https://github.com/saravanan110220/agilegithub.git
 > git.exe --version # timeout=10
 > git --version # 'git version 2.53.0.windows.1'
 > git.exe fetch --tags --force --progress -- https://github.com/saravanan110220/agilegithub.git +refs/heads/*:refs/remotes/origin/* # timeout=10
 > git.exe rev-parse "refs/remotes/origin/main^{commit}" # timeout=10
Checking out Revision cc7504d4c02efb0565a3b43f6f7d17b81e1aef16 (refs/remotes/origin/main)
 > git.exe config core.sparsecheckout # timeout=10
 > git.exe checkout -f cc7504d4c02efb0565a3b43f6f7d17b81e1aef16 # timeout=10
 > git.exe branch -a -v --no-abbrev # timeout=10
 > git.exe checkout -b main cc7504d4c02efb0565a3b43f6f7d17b81e1aef16 # timeout=10
Commit message: "Add files via upload"
First time build. Skipping changelog.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Show Files)
```

Jenkins / Pipeline-Task1 / #2 / Console Output

- Previous Build

```
 > git.exe rev-parse "refs/remotes/origin/main^{commit}" # timeout=10
Checking out Revision cc7504d4c02efb0565a3b43f6f7d17b81e1aef16 (refs/remotes/origin/main)
 > git.exe config core.sparsecheckout # timeout=10
 > git.exe checkout -f cc7504d4c02efb0565a3b43f6f7d17b81e1aef16 # timeout=10
 > git.exe branch -a -v --no-abbrev # timeout=10
 > git.exe checkout -b main cc7504d4c02efb0565a3b43f6f7d17b81e1aef16 # timeout=10
Commit message: "Add files via upload"
First time build. Skipping changelog.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Show Files)
[Pipeline] bat

C:\ProgramData\Jenkins\.jenkins\workspace\Pipeline-Task1>dir
 Volume in drive C is Acer
 Volume Serial Number is B285-D27C

 Directory of C:\ProgramData\Jenkins\.jenkins\workspace\Pipeline-Task1

07-02-2026  00:25    <DIR>          .
07-02-2026  00:25    <DIR>          ..
07-02-2026  00:25               599 fibonacci.c
07-02-2026  00:25               993 Grade calculator.c
07-02-2026  00:25               400 income.html
07-02-2026  00:25               981 web.css
               4 File(s)          2,973 bytes
               2 Dir(s)  104,870,944,768 bytes free
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

```
C:\ProgramData\Jenkins\.jenkins\workspace\Pipeline-Task1>dir
 Volume in drive C is Acer
 Volume Serial Number is B285-D27C

 Directory of C:\ProgramData\Jenkins\.jenkins\workspace\Pipeline-Task1

07-02-2026  00:25    <DIR>          .
07-02-2026  00:25    <DIR>          ..
07-02-2026  00:25               599 fibonacci.c
07-02-2026  00:25               993 Grade calculator.c
07-02-2026  00:25               400 income.html
07-02-2026  00:25               981 web.css
               4 File(s)          2,973 bytes
               2 Dir(s)  104,870,944,768 bytes free
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Figure 3: Console Output of Pipeline Task 1

Result:

The Jenkins Pipeline successfully checked out code from GitHub and displayed the workspace files.

Pipeline Task 2: Print Current Directory

Objective:

To print the current working directory of the Jenkins workspace.

Step 1:

Create new Pipeline job:

->Pipeline-Task2

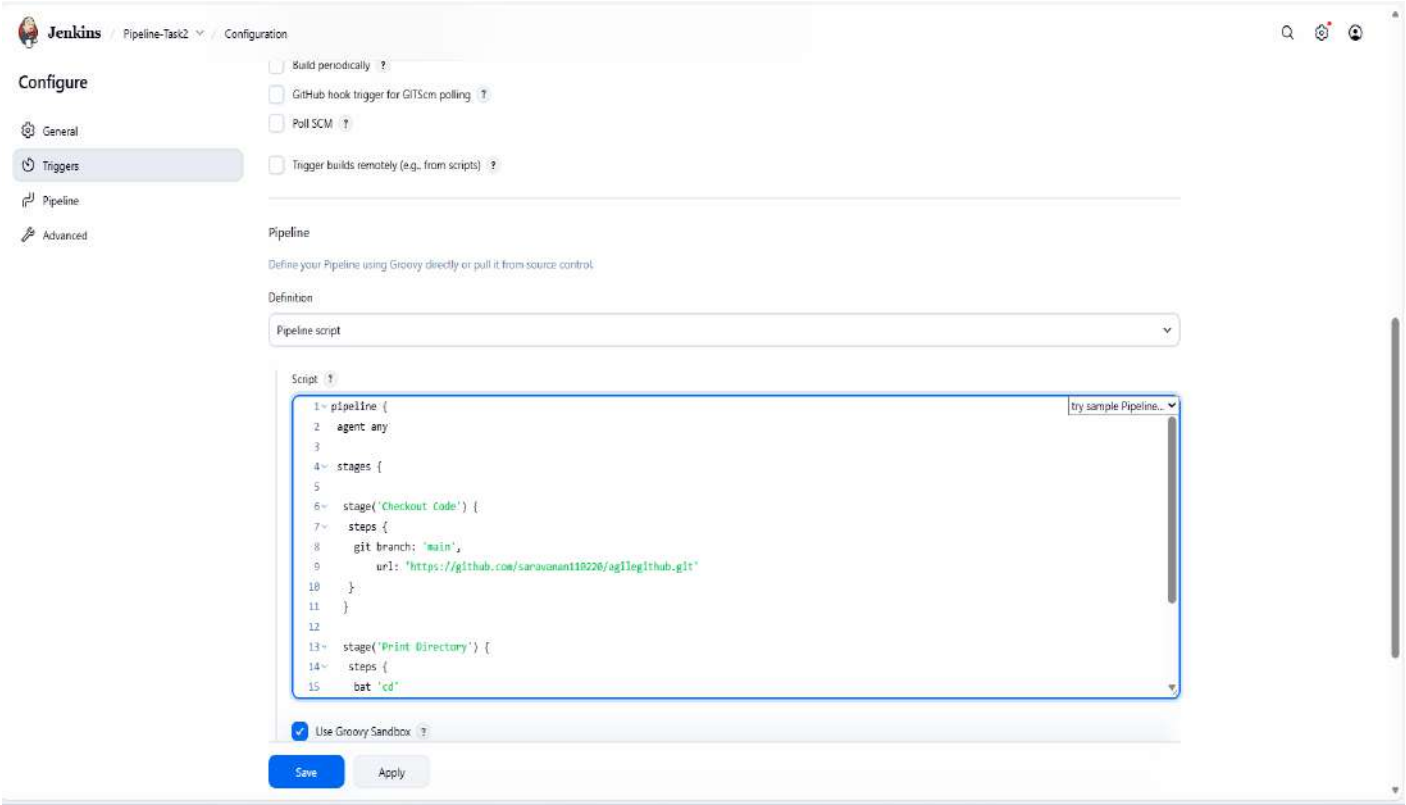
Step 2:

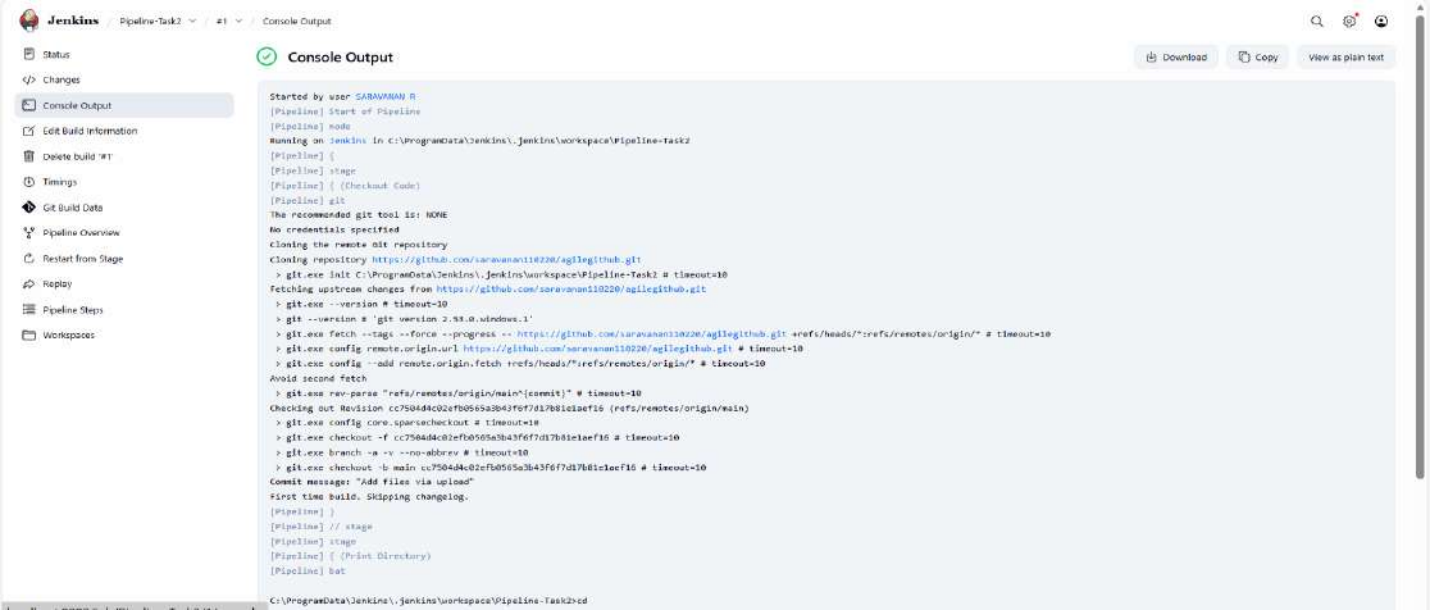
->Enter the following script:

```
pipeline {
agent any

stages {

 stage('Checkout Code') {
```

```
    steps {
     git branch: 'main',
         url: 'https://github.com/saravanan110220/agilegithub.git'
    }
   }

   stage('Print Directory') {
    steps {
     bat 'cd'
    }
   }

  }
 }
```

->Click Save.





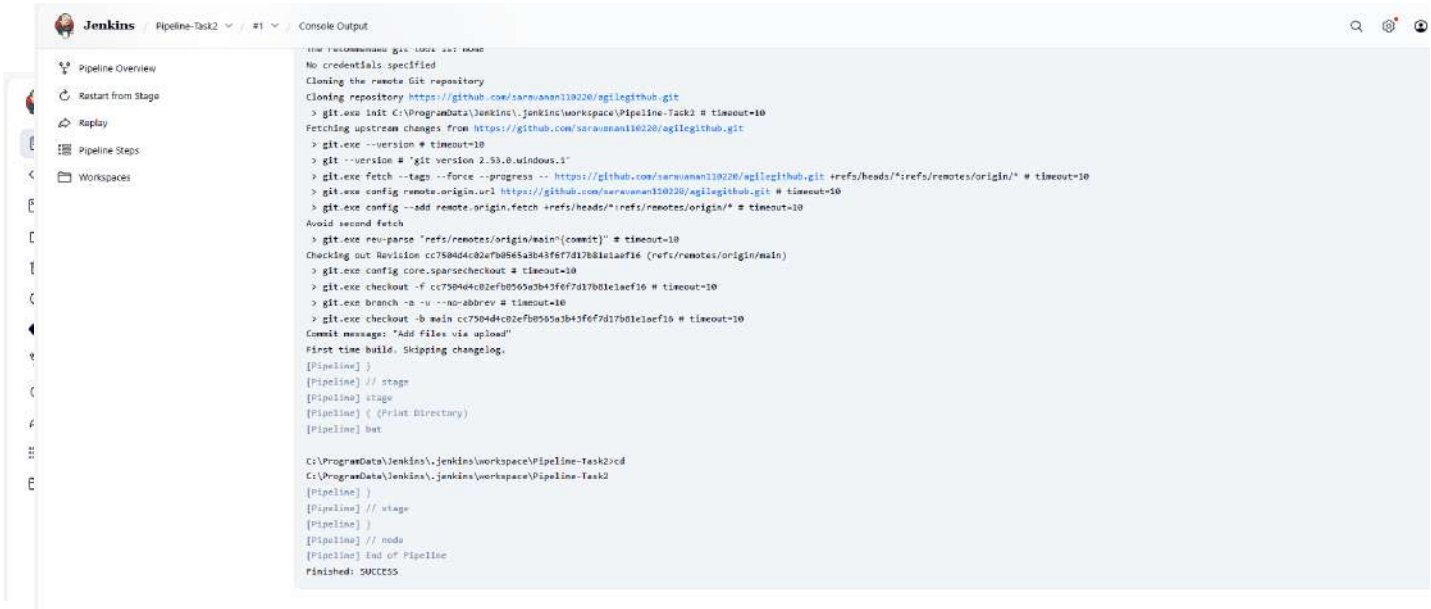


Figure 4: Current Directory Displayed in Pipeline

Result:

The pipeline successfully displayed the current working directory.

◇ Pipeline Task 3: Print Custom Message

Objective :

To display a custom message using Jenkins Pipeline.

Step 1:

->Create job:

Pipeline-Task3

Step 2:

->Add script:

```
 pipeline {
 agent any

 stages {

  stage('Checkout Code') {
   steps {
    git branch: 'main',
        url: 'https://github.com/saravanan110220/agilegithub.git'
   }
  }

  stage('Print Message') {
   steps {
    echo 'Hello! Jenkins Pipeline executed successfully'
   }
  }

 }
}
```

Click Save → Build Now.

Jenkins / Pipeline-Task3 / Configuration

Configure

- General
- Triggers
- Pipeline
- Advanced

Poll SCM

Trigger builds remotely (e.g., from scripts)

Pipeline

Define your Pipeline using Groovy directly or pull it from source control

Definition: Pipeline script

Script

try sample Pipeline...

```
pipeline {
  agent any

  stages {

    stage('Checkout Code') {
      steps {
        git branch: 'main',
            url: 'https://github.com/saravanan110220/agilegithub.git'
      }
    }

    stage('Print Message') {
      steps {
        echo 'Hello! Jenkins Pipeline executed successfully'
```

Use Groovy Sandbox

Pipeline Syntax

Save | Apply

Jenkins / Pipeline-Task3 / #1

- Status
- Changes
- Console Output
- Edit Build Information
- Delete build '#1'
- Timings
- Git Build Data
- Pipeline Overview
- Restart from Stage
- Replay
- Pipeline Steps
- Workspaces

#1 (Feb 7, 2026, 12:42:20 AM)

Add description | Keep this build forever

Started by user SARAVANAN R

Started 9.1 sec ago
Took 3.5 sec

This run spent:

- 13 ms waiting;
- 3.5 sec build duration;
- 3.5 sec total from scheduled to completion.

git
Revision: cc7504d4c02efb0565a3b43f6f7d17b81e1aef16
Repository: https://github.com/saravanan110220/agilegithub.git

- refs/remotes/origin/main

No changes.

Jenkins / Pipeline-Task3 / #1 / Console Output

- Status
- Changes
- Console Output
- Edit Build Information
- Delete build '#1'
- Timings
- Git Build Data
- Pipeline Overview
- Restart from Stage
- Replay
- Pipeline Steps
- Workspaces

Console Output

Download | Copy | View as plain text

```
Started by user SARAVANAN R
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in C:\ProgramData\Jenkins\.jenkins\workspace\Pipeline-Task3
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Checkout Code)
[Pipeline] git
The recommended git tool is: NONE
No credentials specified
Cloning the remote Git repository
Cloning repository https://github.com/saravanan110220/agilegithub.git
 > git.exe init C:\ProgramData\Jenkins\.jenkins\workspace\Pipeline-Task3 # timeout=10
Fetching upstream changes from https://github.com/saravanan110220/agilegithub.git
 > git.exe --version # timeout=10
 > git --version # 'git version 2.53.0.windows.1'
 > git.exe fetch --tags --force --progress -- https://github.com/saravanan110220/agilegithub.git +refs/heads/*:refs/remotes/origin/* # timeout=10
 > git.exe config remote.origin.url https://github.com/saravanan110220/agilegithub.git # timeout=10
 > git.exe config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
 > git.exe rev-parse "refs/remotes/origin/main^{commit}" # timeout=10
Checking out Revision cc7504d4c02efb0565a3b43f6f7d17b81e1aef16 (refs/remotes/origin/main)
 > git.exe config core.sparsecheckout # timeout=10
 > git.exe checkout -f cc7504d4c02efb0565a3b43f6f7d17b81e1aef16 # timeout=10
 > git.exe branch -a -v --no-abbrev # timeout=10
 > git.exe checkout -b main cc7504d4c02efb0565a3b43f6f7d17b81e1aef16 # timeout=10
Commit message: "Add files via upload"
First time build. Skipping changelog.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Print Message)
[Pipeline] echo
Hello! Jenkins Pipeline executed successfully
[Pipeline] }
[Pipeline] // stage
```

```
 > git.exe branch -a -v --no-abbrev # timeout=10
 > git.exe checkout -b main cc7504d4c02efb0565a3b43f6f7d17b81e1aef16 # timeout=10
Commit message: "Add files via upload"
First time build. Skipping changelog.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Print Message)
[Pipeline] echo
Hello! Jenkins Pipeline executed successfully
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Figure 5: Custom Message Printed in Pipeline

Result:

The pipeline successfully printed the specified message

- **Pipeline Task 4: Create a File**

Objective:

To create a text file inside Jenkins workspace using Pipeline.

Step 1:

->Create job:

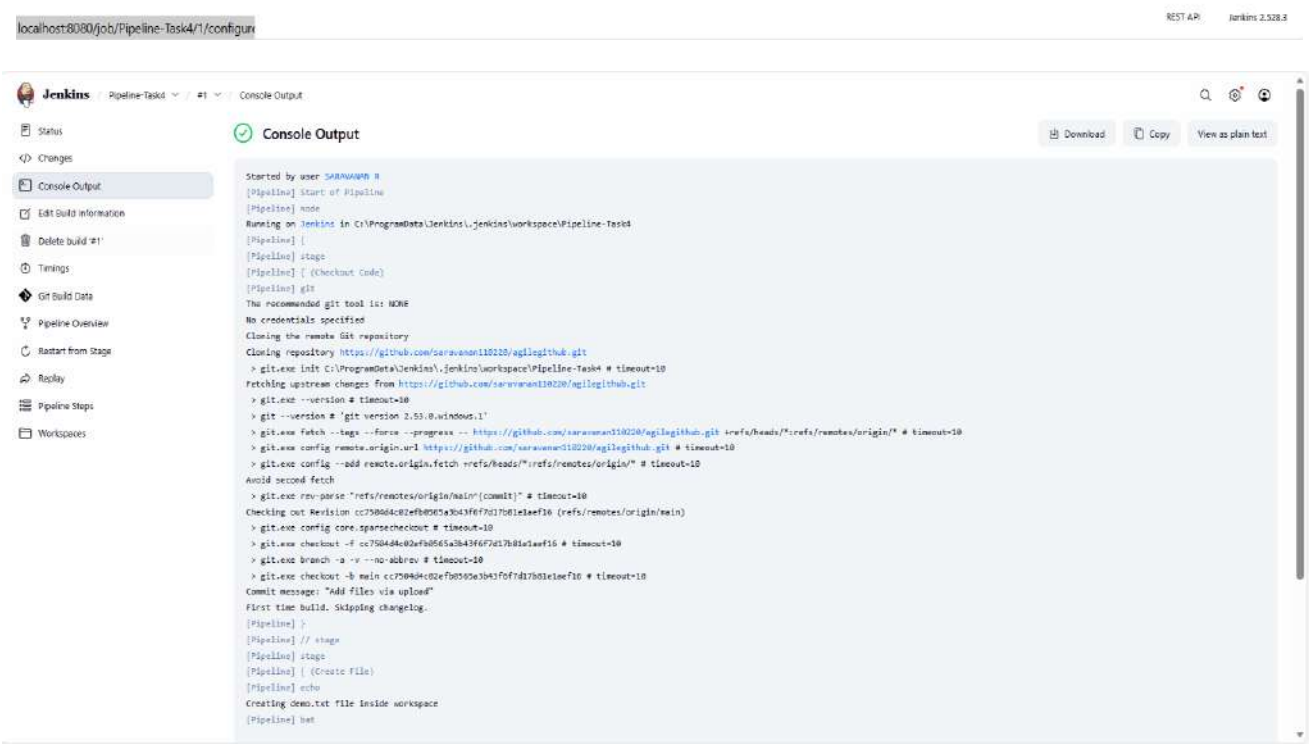
Pipeline-Task4

Step 2:

->Add script:

```
pipeline {
 agent any

 stages {

  stage('Checkout Code') {
   steps {
    git branch: 'main',
        url: 'https://github.com/saravanan110220/agilegithub.git'
   }
  }
```

```
    stage('Create File') {
     steps {
      echo "Creating demo.txt file inside workspace"
      bat 'echo This file is created by Jenkins > demo.txt'
     }
    }


  }
 }
```



```
 > git.exe rev-parse refs/remotes/origin/main^{commit} # timeout=10
Checking out Revision cc7504d4c02efb0565a3b43f6f7d17b81e1aef16 (refs/remotes/origin/main)
 > git.exe config core.sparsecheckout # timeout=10
 > git.exe checkout -f cc7504d4c02efb0565a3b43f6f7d17b81e1aef16 # timeout=10
 > git.exe branch -a -v --no-abbrev # timeout=10
 > git.exe checkout -b main cc7504d4c02efb0565a3b43f6f7d17b81e1aef16 # timeout=10
Commit message: "Add files via upload"
First time build. Skipping changelog.
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Create File)
[Pipeline] echo
Creating demo.txt file inside workspace
[Pipeline] bat

C:\ProgramData\Jenkins\.jenkins\workspace\Pipeline-Task4>echo This file is created by Jenkins  1>demo.txt
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

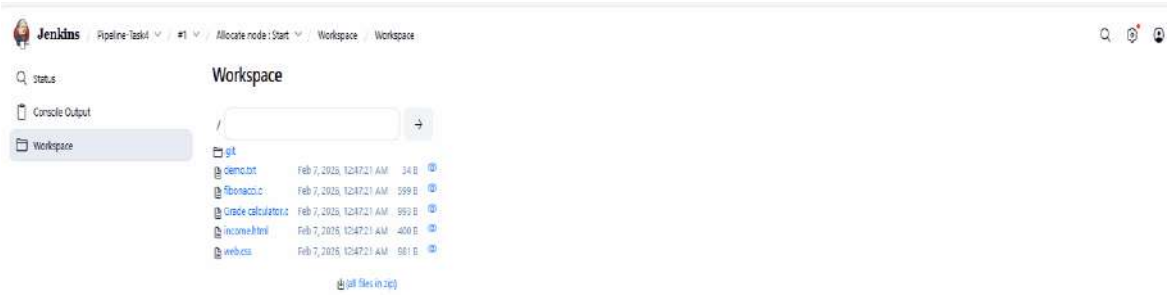


Figure 6: File Created Successfully in Workspace

Result :

The pipeline successfully created a text file in the Jenkins workspace.

- **Pipeline Task 5: Read File Content**

Objective :

To read and display contents of a file using Jenkins Pipeline.

Step 1:

->Create job:

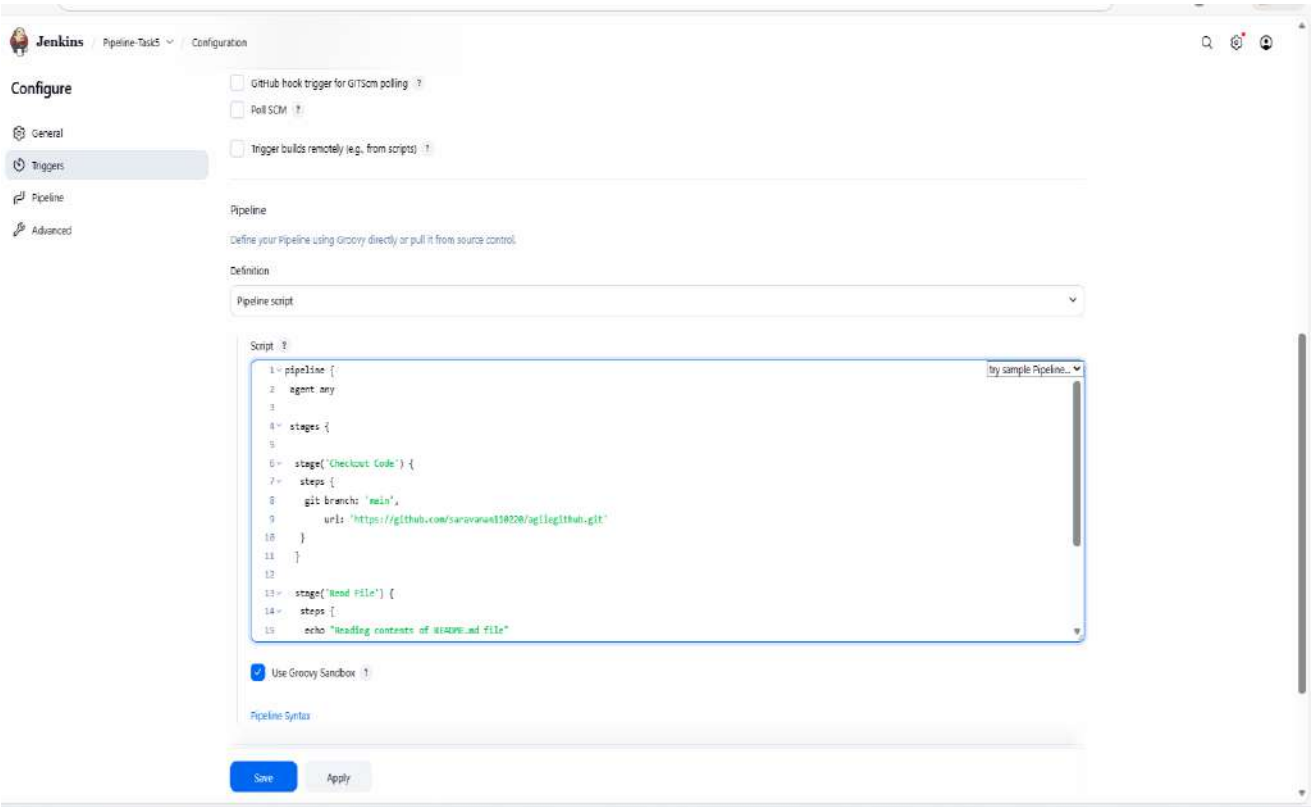
Pipeline-Task5

Step 2:

->Add script:

```
pipeline {
 agent any

 stages {
```

```
stage('Checkout Code') {
 steps {
  git branch: 'main',
      url: 'https://github.com/saravanan110220/agilegithub.git'
 }
}

stage('Read File') {
 steps {
  echo "Reading contents of README.md file"
  bat 'type README.md'
 }
}

}
}
```



Jenkins / Pipeline-Task5 / #2

- Status
- Changes
- Console Output
- Edit Build Information
- Delete build '#2'
- Timings
- Git Build Data
- Pipeline Overview
- Restart from Stage
- Replay
- Pipeline Steps
- Workspaces
- Previous Build

#2 (Feb 7, 2026, 12:59:46 AM)

Add description | Keep this build forever

Started by user SARAVANAN R

Started 14 sec ago
Took 2.9 sec

This run spent:

- 8 ms waiting;
- 2.9 sec build duration;
- 2.9 sec total from scheduled to completion.

Revision: d91de4d7db7ab6e67ce909d670d225800416b548
Repository: https://github.com/saravanan110220/agilegithub.git

- refs/remotes/origin/main

Changes

1. Add files via upload (details / github.com)

Jenkins / Pipeline-Task5 / #2 / Console Output

- Status
- Changes
- Console Output
- Edit Build Information
- Delete build '#2'
- Timings
- Git Build Data
- Pipeline Overview
- Restart from Stage
- Replay
- Pipeline Steps
- Workspaces
- Previous Build

Console Output

Download | Copy | View as plain text

```
Started by user SARAVANAN R
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in C:\ProgramData\Jenkins\.jenkins\workspace\Pipeline-Task5
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Checkout Code)
[Pipeline] git
The recommended git tool is: NONE
No credentials specified
 > git.exe rev-parse --resolve-git-dir C:\ProgramData\Jenkins\.jenkins\workspace\Pipeline-Task5\.git # timeout=10
Fetching changes from the remote Git repository
 > git.exe config remote.origin.url https://github.com/saravanan110220/agilegithub.git # timeout=10
Fetching upstream changes from https://github.com/saravanan110220/agilegithub.git
 > git.exe --version # timeout=10
 > git --version # 'git version 2.53.0.windows.1'
 > git.exe fetch --tags --force --progress -- https://github.com/saravanan110220/agilegithub.git +refs/heads/*:refs/remotes/origin/* # timeout=10
 > git.exe rev-parse "refs/remotes/origin/main^{commit}" # timeout=10
Checking out Revision d91de4d7db7ab6e67ce909d670d225800416b548 (refs/remotes/origin/main)
 > git.exe config core.sparsecheckout # timeout=10
 > git.exe checkout -f d91de4d7db7ab6e67ce909d670d225800416b548 # timeout=10
 > git.exe branch -a -v --no-abbrev # timeout=10
 > git.exe branch -D main # timeout=10
 > git.exe checkout -b main d91de4d7db7ab6e67ce909d670d225800416b548 # timeout=10
Commit message: "Add files via upload"
 > git.exe rev-list --no-walk cc7504d4c02efb0565a3b43f6f7d17b81e1aef16 # timeout=10
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Read File)
[Pipeline] echo
Reading contents of README.md file
[Pipeline] bat

C:\ProgramData\Jenkins\.jenkins\workspace\Pipeline-Task5>type README.md
Git is a powerful, open‚Äësource version control system that helps developers track changes in code, collaborate across teams, and maintain a complete history of their projects. It‚Äôs the backbone of
```

localhost:8080

Jenkins / Pipeline-Task5 / #2 / Console Output

- Previous Build

```
 > git.exe rev-parse "refs/remotes/origin/main^{commit}" # timeout=10
Checking out Revision d91de4d7db7ab6e67ce909d670d225800416b548 (refs/remotes/origin/main)
 > git.exe config core.sparsecheckout # timeout=10
 > git.exe checkout -f d91de4d7db7ab6e67ce909d670d225800416b548 # timeout=10
 > git.exe branch -a -v --no-abbrev # timeout=10
 > git.exe branch -D main # timeout=10
 > git.exe checkout -b main d91de4d7db7ab6e67ce909d670d225800416b548 # timeout=10
Commit message: "Add files via upload"
 > git.exe rev-list --no-walk cc7504d4c02efb0565a3b43f6f7d17b81e1aef16 # timeout=10
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Read File)
[Pipeline] echo
Reading contents of README.md file
[Pipeline] bat

C:\ProgramData\Jenkins\.jenkins\workspace\Pipeline-Task5>type README.md
Git is a powerful, open‚Äësource version control system that helps developers track changes in code, collaborate across teams, and maintain a complete history of their projects. It‚Äôs the backbone of
modern software development, used with platforms like GitHub, GitLab, and Bitbucket to share and manage code.

Ôÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿÿ
Key Points About Git
Version Control: Git records every change made to files, allowing you to roll back to earlier versions if needed.

Distributed System: Unlike centralized systems, every developer has a full copy of the repository, including its history.

Collaboration: Multiple developers can work on the same project simultaneously, merging their contributions efficiently.

Branching & Merging: Git makes it easy to create branches for new features or experiments, then merge them back into the main project.

Speed & Efficiency: Operations like commits, diffs, and merges are optimized for performance.
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Figure 7: File Content Displayed in Pipeline

Result:

The pipeline successfully read and displayed the contents of the specified file.

EXPERIMENT – 6 :

Jenkins CI Pipeline using Git and Maven

Objective:

To implement a Continuous Integration (CI) pipeline using Jenkins that automatically checks out a Maven project from GitHub, builds the project, runs JUnit test cases, and reports the build status.

Tools Used:

- Jenkins
- Git
- GitHub
- Apache Maven
- Java JDK
- Web Browser

Step 1: Create a Simple Maven Project (Git Side)

Project Name:

simple-maven-app

Standard Maven Folder Structure (Mandatory):

```
simple-maven-app/
 ├── pom.xml
 └── src/
    ├── main/java/com/example/App.java
    └── test/java/com/example/AppTest.java
```

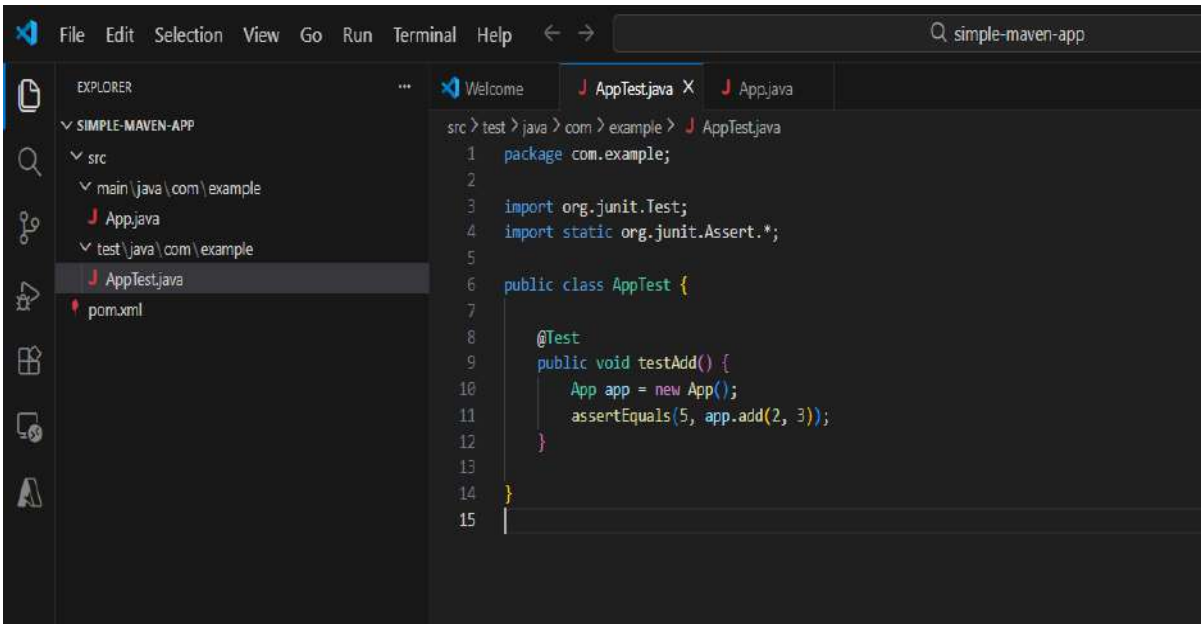


Figure 1: Maven Standard Project Structure

<u>Step 2:</u> Create App.java (Main Application)

File Path:

src/main/java/com/example/App.java

<u>Code:</u>

```
package com.example;


public class App {
    public int add(int a, int b) {
        return a + b;
    }
}
```

Explanation:

- Defines business logic.
- add() method returns sum of two numbers.
- This method will be tested using JUnit.

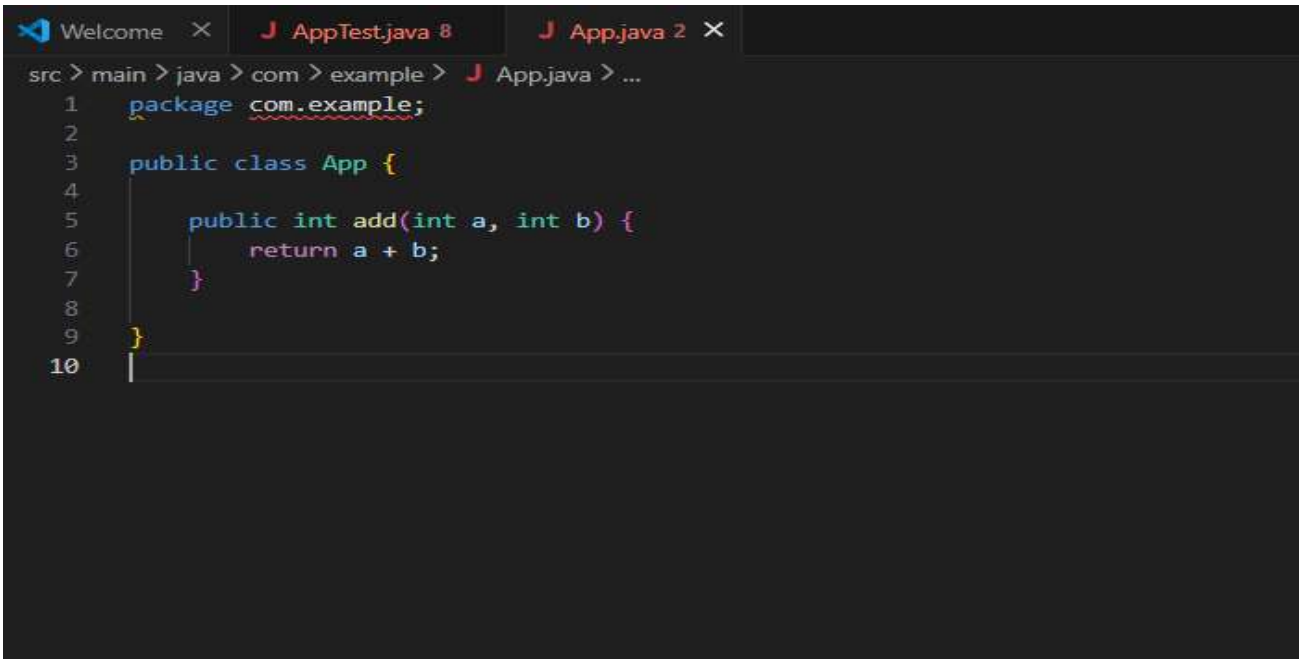


Figure 2: Main Application File (App.java)

<u>Step 3:</u> Create AppTest.java (JUnit Test Case)

File Path:

src/test/java/com/example/AppTest.java

<u>Code:</u>

```
package com.example;


import org.junit.Test;

import static org.junit.Assert.*;


public class AppTest {


    @Test

    public void testAdd() {

        App app = new App();

        assertEquals(5, app.add(2, 3));

    }

}
```

Explanation:

- @Test marks the method as a test case.
- assertEquals() compares expected and actual values.
- If test fails → Maven build fails.
- If test passes → Build success.

```
Welcome    AppTest.java 8
src > test > java > com > example > AppTest.java > ...
1   package com.example;
2
3   import org.junit.Test;
4   import static org.junit.Assert.*;
5
6   public class AppTest {
7
8       @Test
9       public void testAdd() {
10          App app = new App();
11          assertEquals(5, app.add(2, 3));
12      }
13
14  }
15
```

Figure 3: JUnit Test Case Implementation

<u>Step 4:</u> Create pom.xml

File Path:

simple-maven-app/pom.xml

<u>Code:</u>

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
 xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
 http://maven.apache.org/xsd/maven-4.0.0.xsd">

 <modelVersion>4.0.0</modelVersion>
 <groupId>com.example</groupId>
 <artifactId>simple-maven-app</artifactId>
 <version>1.0-SNAPSHOT</version>

 <dependencies>
  <dependency>
   <groupId>junit</groupId>
   <artifactId>junit</artifactId>
   <version>4.13.2</version>
   <scope>test</scope>
  </dependency>
 </dependencies>

</project>
```

Explanation:

- pom.xml defines project configuration.
- JUnit dependency enables unit testing.
- Maven uses this file to build the project.

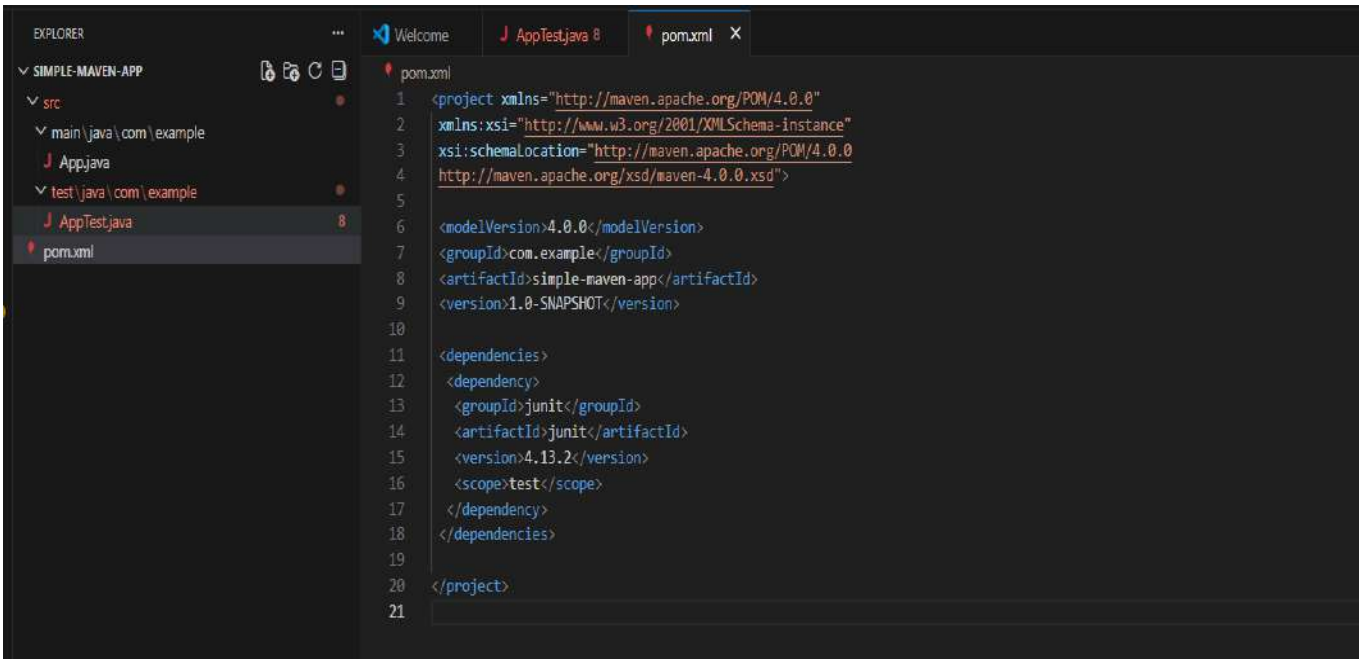


Figure 4: Maven Configuration File (pom.xml)

<u>Step 5:</u> Test Project Locally

Open terminal inside project folder:

mvn clean test

<u>Expected Output:</u>

BUILD SUCCESS

Figure 5: Local Maven Build Successful

<u>Step 6:</u> Push Project to GitHub

COMMANDS:

```
git init

git add README.md

git commit -m "first commit"

git branch -M main

git remote add origin https://github.com/saravanan110220/maven-pipeline.git

git push -u origin main
```

```
PROBLEMS 2   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   AZURE

C:\Users\sarva\Downloads\simple-maven-app>
C:\Users\sarva\Downloads\simple-maven-app>maven-pipeline
'maven-pipeline' is not recognized as an internal or external command,
operable program or batch file.

C:\Users\sarva\Downloads\simple-maven-app>
C:\Users\sarva\Downloads\simple-maven-app>git remote add origin https://github.com/saravanan110220/maven-pipeline.git

C:\Users\sarva\Downloads\simple-maven-app>
C:\Users\sarva\Downloads\simple-maven-app>git push -u origin main
Enumerating objects: 36, done.
Counting objects: 100% (36/36), done.
Delta compression using up to 12 threads
Compressing objects: 100% (16/16), done.
Writing objects: 100% (36/36), 4.08 KiB | 418.00 KiB/s, done.
Total 36 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/saravanan110220/maven-pipeline.git
 * [new branch]      main -> main
branch 'main' set up to track 'origin/main'.

C:\Users\sarva\Downloads\simple-maven-app>
C:\Users\sarva\Downloads\simple-maven-app>
```

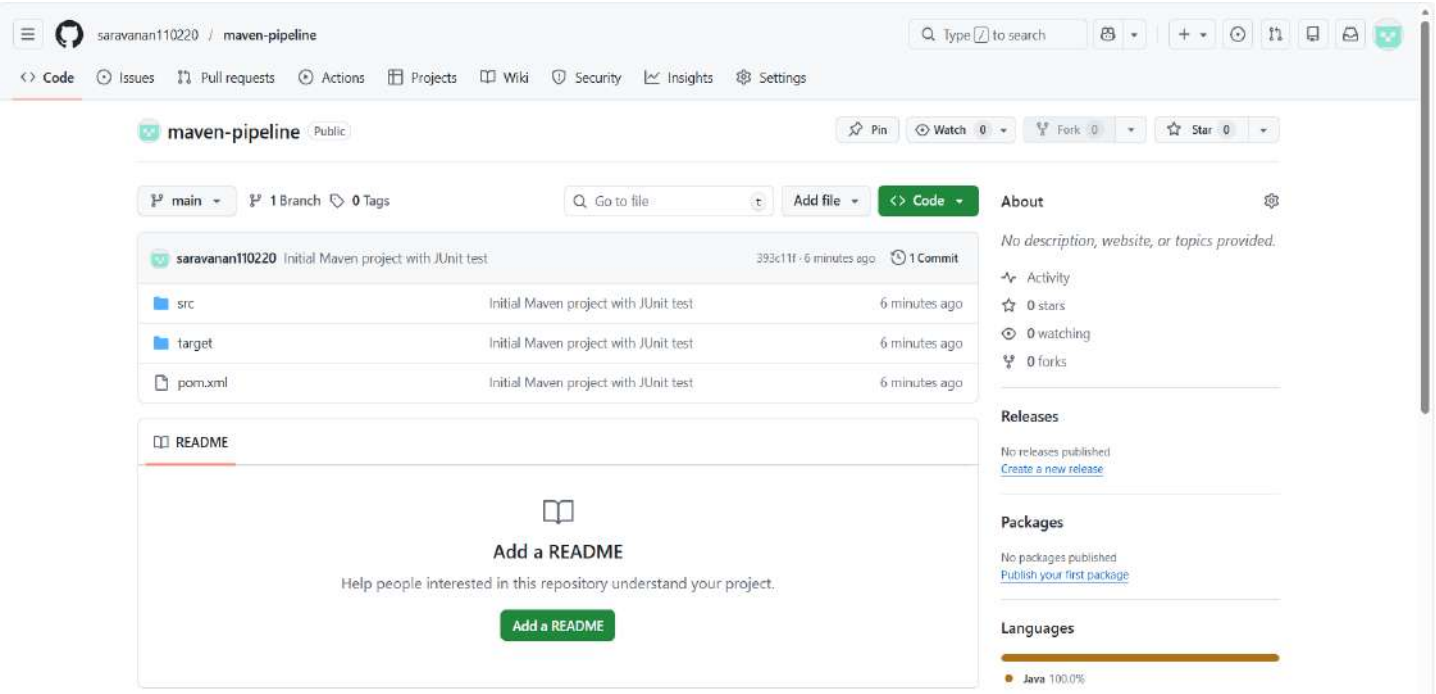


Figure 6: Maven Project Uploaded to GitHub

<u>Step 7:</u> Install Maven on Windows

1. Download Apache Maven.
2. **Extract to:**

   C:\Program Files\Apache\apache-maven-3.9.12
3. **Create System Variable:**

   MAVEN_HOME = C:\Program Files\Apache\apache-maven-3.9.12
4. **Add to PATH:**

   %MAVEN_HOME%\bin
5. Verify:

   mvn -version

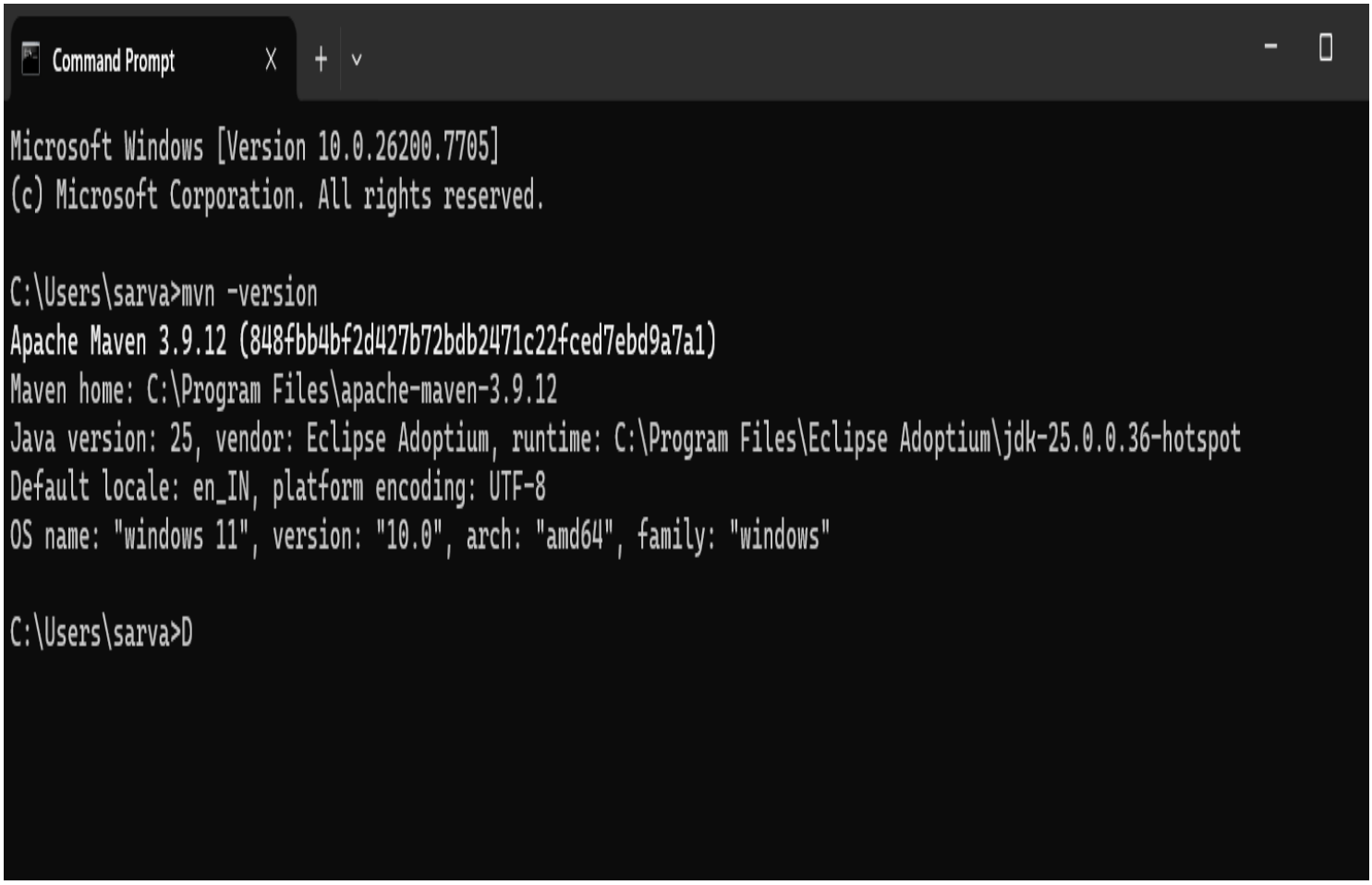


Figure 7: Maven Installed Successfully

<u>Step 8:</u> Configure Maven in Jenkins

Go to:

Manage Jenkins → Tools → Maven Installations

<u>Configure:</u>

- Name: M3
- MAVEN_HOME: C:\Program Files\Apache\apache-maven-3.9.12
- Uncheck Install Automatically

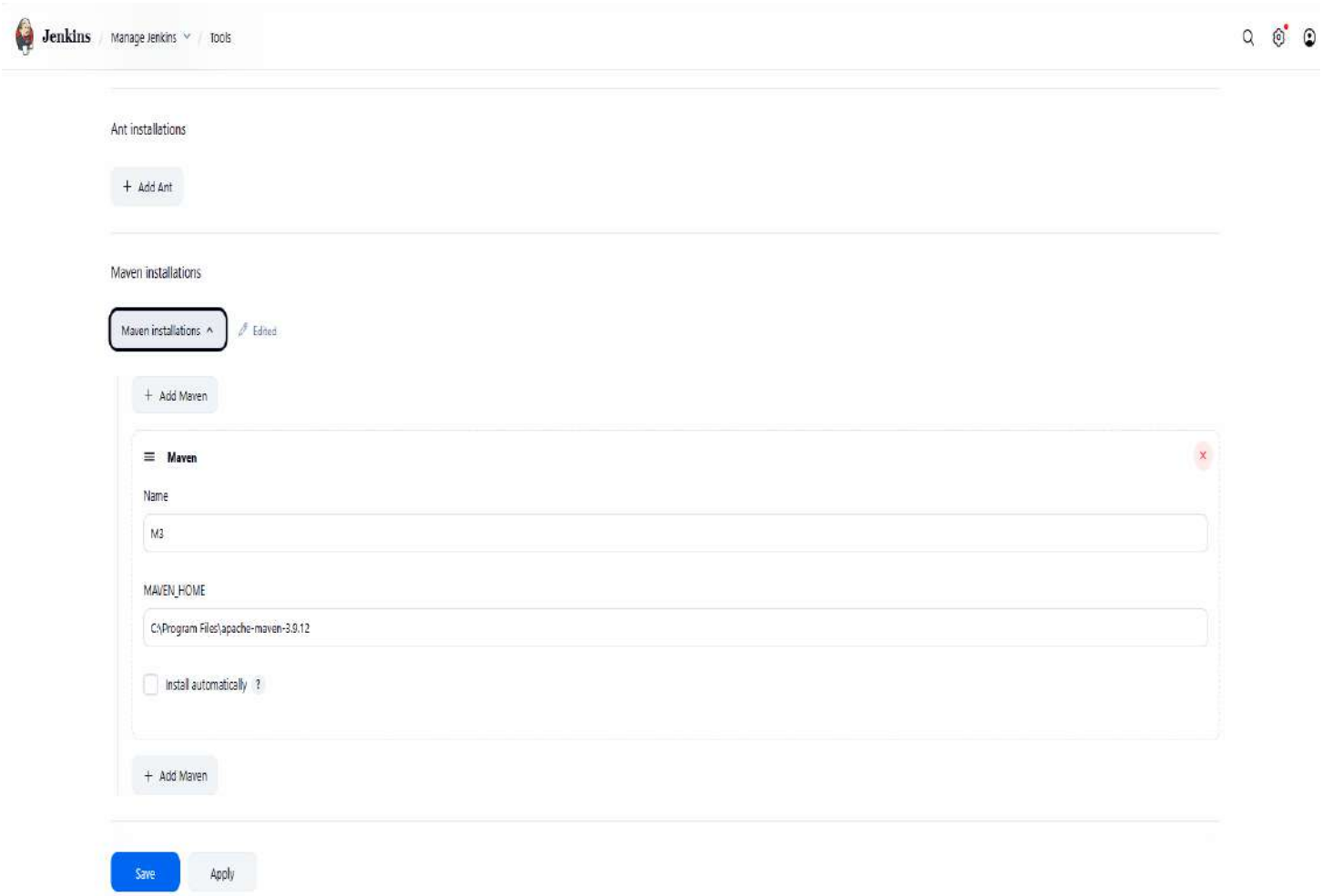


Figure 8: Maven Configured in Jenkins

<u>Step 9:</u> Create Jenkins Pipeline

Create new Pipeline job.

Pipeline Script:

```
pipeline {
    agent any

    tools {
        maven 'M3'
        // 'M3' must match the Maven name you configured in Jenkins Tools
    }

    stages {
        stage('Checkout Source Code') {
            steps {
                git branch: 'main',
                    url: 'https://github.com/saravanan110220/maven-pipeline.git'
            }
        }

        stage('Build & Run Tests') {
            steps {
                echo "Running Maven build and executing JUnit tests..."
                bat 'mvn clean test'
            }
        }
    }

    post {
        success {
            echo "Pipeline completed successfully!"
        }
        failure {
            echo "Pipeline failed. Check errors above!"
```

```
        }
    }
}
```

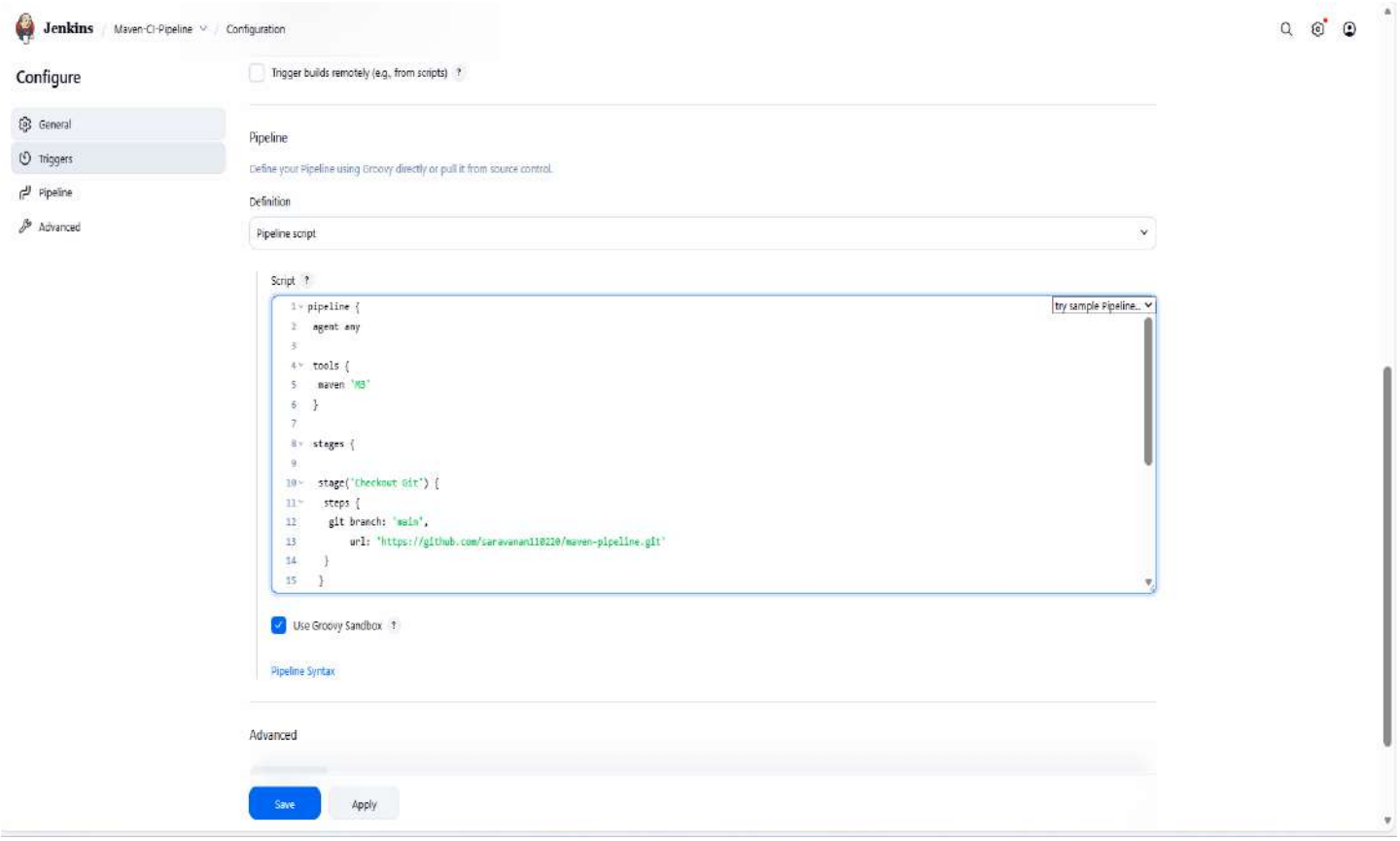
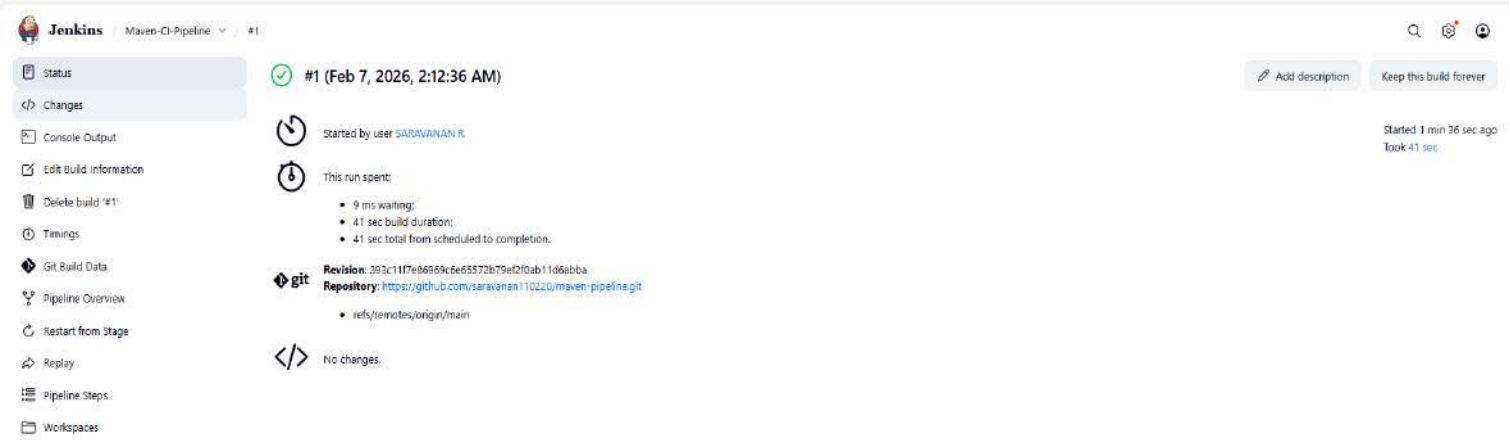


Figure 9: Jenkins Declarative Pipeline Script

<u>Step 10:</u> Run Jenkins Pipeline

1. Click Save.
2. Click Build Now.
3. Open Console Output.



Jenkins / Maven-CI-Pipeline / #1 / Console Output

Status
Changes
Console Output
Edit Build Information
Delete build '#1'
Timings
Git Build Data
Pipeline Overview
Restart from Stage
Replay
Pipeline Steps
Workspaces

Console Output

Download | Copy | View as plain text

```
Started by user SARAVANAN R
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in C:\ProgramData\Jenkins\.jenkins\workspace\Maven-CI-Pipeline
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Tool Install)
[Pipeline] tool
[Pipeline] envVarsForTool
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Checkout Git)
[Pipeline] tool
[Pipeline] envVarsForTool
[Pipeline] withEnv
[Pipeline] {
[Pipeline] git
The recommended git tool is: NONE
No credentials specified
Cloning the remote Git repository
Cloning repository https://github.com/saravanan110220/maven-pipeline.git
 > git.exe init C:\ProgramData\Jenkins\.jenkins\workspace\Maven-CI-Pipeline # timeout=10
Fetching upstream changes from https://github.com/saravanan110220/maven-pipeline.git
 > git.exe --version # timeout=10
 > git --version # 'git version 2.53.0.windows.1'
 > git.exe fetch --tags --force --progress -- https://github.com/saravanan110220/maven-pipeline.git +refs/heads/*:refs/remotes/origin/* # timeout=10
 > git.exe config remote.origin.url https://github.com/saravanan110220/maven-pipeline.git # timeout=10
 > git.exe config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
 > git.exe rev-parse "refs/remotes/origin/main^{commit}" # timeout=10
Checking out Revision 393c11f7e86969c6e65572b79ef2f0ab11d6abba (refs/remotes/origin/main)
 > git.exe config core.sparsecheckout # timeout=10
 > git.exe checkout -f 393c11f7e86969c6e65572b79ef2f0ab11d6abba # timeout=10
```

Jenkins / Maven-CI-Pipeline / #1 / Console Output

```
Progress (3): 12 kB | 18 kB | 7.7/26 kB
Progress (3): 12 kB | 18 kB | 16/26 kB
Progress (4): 12 kB | 18 kB | 16/26 kB | 7.7/18 kB
Progress (4): 12 kB | 18 kB | 16/26 kB | 16/18 kB
Progress (4): 12 kB | 18 kB | 16/26 kB | 18 kB
Progress (4): 12 kB | 18 kB | 26 kB | 18 kB

Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/common-junit3/3.2.5/common-junit3-3.2.5.jar (12 kB at 33 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/surefire-junit4/3.2.5/surefire-junit4-3.2.5.jar (18 kB at 41 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/common-java5/3.2.5/common-java5-3.2.5.jar (18 kB at 37 kB/s)
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/surefire/common-junit4/3.2.5/common-junit4-3.2.5.jar (26 kB at 42 kB/s)
[INFO]
[INFO] -------------------------------------------------------
[INFO]  T E S T S
[INFO] -------------------------------------------------------
[INFO] Running com.example.AppTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.053 s -- in com.example.AppTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  35.806 s
[INFO] Finished at: 2026-02-07T02:13:16+05:30
[INFO] ------------------------------------------------------------------------
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Figure 10: Successful Jenkins CI Pipeline Execution

Expected:

- Tests run successfully

- BUILD SUCCESS

– Finished: SUCCESS

FINAL RESULT:

- Jenkins checks out the Maven project from GitHub, builds it using Maven, runs JUnit unit tests automatically, and reports build success or failure.
- This completes a full CI pipeline.